

Fiat-Shamir Goes Rational

(Or: On the Perils of Sublinear Verification)

Matteo Campanelli¹  and Agni Datta^{2*} 

¹ Offchain Labs, Inc.

[\[binarywhalesinternaryseas@gmail.com\]](mailto:binarywhalesinternaryseas@gmail.com)

² No affiliation

[\[agnidatta.org@gmail.com\]](mailto:agnidatta.org@gmail.com)

Abstract We investigate the existence of non-interactive proof systems where the verifier runs in time sublinear in the input size. This seemingly paradoxical property has been shown possible in interactive settings for specific soundness notions, in particular proofs of proximity (Rothblum, Vadhan, and Wigderson, STOC 2013) and rational proofs (Azar and Micali, STOC 2012).

We initiate the study of the Fiat-Shamir paradigm in the sublinear verification setting. We show Fiat-Shamir to generally fail in this context, and, importantly, we study *how* it fails. Our results include:

- **General impossibility results from properties of Boolean functions.** We introduce an abstract framework that generically applies to the sublinear setting (regardless of the soundness notion). Our framework includes an appropriately adapted variant of Fiat-Shamir, as well as general sufficient conditions under which employing Fiat-Shamir yields insecure protocols. Motivated by their practical interest and simplicity, we then apply our framework to rational proofs. We develop attack strategies that leverage the sublinearity of the verifier and exploit complexity metrics of Boolean functions, revealing an intriguing connection.
- **Robustness and applicability of our attacks.** We ask whether a strengthened variant of Fiat-Shamir—which includes even parts of the public inputs that the verifier has not queried—can yield secure protocols. We show negative results in this setting as well. In particular, we prove that representative protocols for broad and “easy” function classes—shallow Boolean circuits (Campanelli and Gennaro, 2015) and threshold circuits (Azar and Micali, 2013)—become insecure when compiled with either of our Fiat-Shamir transforms.

Our general results provide further insights into non-interactive sublinear verification, complementing the concurrent work by Chen, Jin, and Wichs (STOC 2025), which shows lower bounds for non-interactive proofs of proximity for a specific problem within P.

* Work done in part while the author was affiliated with VIT Bhopal University and during his internship at the CISA Helmholtz Center for Information Security.

Contents

1	Introduction	3
1.1	Our Contributions in a Nutshell	5
1.2	Discussion and Open Questions	7
1.3	Related Work	11
2	Technical Overview	11
3	Background on Rational Arguments	16
3.1	Preliminaries and Notation	16
3.2	Interactive Rational Arguments	17
3.3	Non-Interactive Rational Arguments in ROM	19
4	Fiat-Shamir Transforms in the Sublinear Setting and TTF Attacks	20
4.1	Generic Holographic Arguments	20
4.2	Input Query Sets and Safe Sets	21
4.3	A Fiat-Shamir Transform for Holographic Protocols (hFS)	22
4.4	True-to-False-Input Attacks in hFS	24
4.5	TTF Adversaries Break Fiat-Shamir	24
4.6	Semi-Adaptive Soundness and its Vulnerability to TTF	26
4.7	A Stronger Fiat-Shamir Compiler with “Input Digests”	27
5	Generic TTF Attacks from Boolean Function Properties	28
5.1	Preliminaries on Boolean Functions	29
5.2	Attacks from Impressionable Functions	31
5.3	Attacks from σ -Sensitivity	31
6	Concrete TTF Attacks Against Rational Arguments	32
6.1	Background on Our Case Studies	32
6.2	TTF Attacks Against [CG15] and [AM13]	35
7	Further Attacks Against Fiat-Shamir in the Input-Digest Model	38
7.1	Negative Results for AM13	38
7.2	Negative Results for CG15	41
	References	44
A	Prior Definitions of Rational Proofs and Arguments	48
A.1	Definition Based on Reward Gap	48
A.2	Quantifying Adversarial Loss: Reward Gap	48
A.3	Equivalence Between Models	49
B	Rational Arguments in the Input-Digest Model	51
C	Standard Fiat-Shamir Transform	52
D	Fiat-Shamir Transform for Proofs of Proximity	53
D.1	Property Testing and Proximity Arguments	53
D.2	Arguments of Proximity	54
D.3	The Fiat-Shamir Transform in the Arguments of Proximity Model	56
D.4	Safe Sets in Arguments of Proximity	57
E	The Synergy Rational Proofs $\longleftrightarrow$ Smart Contracts	58

1 Introduction

A (cryptographic) proof system [Mic94, Tha22a] is a protocol enabling an untrusted server to convince a client of the veridicity of a statement. A typical statement would claim, for instance, that a computation f executed on the client’s data x produces the output y (concretely, f might correspond to a machine learning training algorithm, x a training dataset, and y the resulting model). In this setting—known as *verifiable computation* [WB15]—the client should be able to check the server’s claim without having to recompute $f(x)$ from scratch, something that could be infeasible due to resource constraints or high costs. Such protocols may be interactive, with the server (the *prover*) and the client (the *verifier*) jointly generating a transcript τ that the latter can check efficiently. If the server is lying and claims a false output, the discrepancy is almost certainly detected via τ .

Often, it is desirable for such a protocol to be *non-interactive*, meaning that once the client receives the *certificate*, it should be able to validate it independently, without any further interaction with the server. A common technique to achieve this non-interactivity is the well-known *Fiat-Shamir transform* [FS87]. In this approach, whenever the (public-coin) verifier sends a random challenge to the prover, the prover instead computes the challenge itself. Specifically, each challenge e_i is computed as $e_i = H(\text{pp}, x, \tau_{i-1})$, where pp are public parameters, x is the input, and τ_{i-1} is the transcript up to that point. For security purposes, the hash function H is often modeled as a random oracle (RO)³

The security of the Fiat-Shamir transform has been widely studied in the context of standard interactive proofs [FS87, PS96], with critical contributions tracing back to works like [Bar01, GK03], and more recently [KRS25]. However, several key papers have also shown what happens when the Fiat-Shamir transform is not applied correctly [DMWG23]. For example, there are well-known vulnerabilities tied to the so-called *weak-FS* paradigm⁴ where some (or all) of the public inputs are omitted from the challenge generation. The idea behind these attacks is that, since not all of x is fed into the hash, the adversary can manipulate the transcript to “fit” a false input or one for which it does not have a valid witness.

To understand the risks involved, consider the difference between a “proper” application of Fiat-Shamir, one that uses the full public input, and the *weak-FS* scenario. If a transcript τ is generated without using the verifier’s full public input, the verifier might face a situation where the prover can *adaptively* find a specific statement $\hat{x}$ (for which the original claim might be false) and still have $\text{Verify}(\text{claim}, \hat{x}, \tau)$ accept. In other words, a significant threat to the verifier in *weak-FS* comes from the fact that not all of the verifier’s view, which includes the public input, is involved in the challenge generation process.

This Work: Fiat-Shamir and Sublinear Verification. The starting point of this work is a natural question on the relationship between the view of the verifier and Fiat-Shamir. Specifically, we are inspired by the existence of proof systems such as proofs of proximity [GGR15, RVW13, IW14, GR17, RR20, CG18b, GR22, GRS23] and rational proofs [AM12, GHRV14, CG15, GHRV16, IY17, ABDY24] where the verifier runs in time sublinear in the size of its input. This class of

³ There are alternative instantiations in the literature to overcome the limitations of assuming the existence of a random oracle (see [CGH98, CCR16]). In this work, we focus exclusively on Fiat-Shamir instantiated with a RO.

⁴ This is often an “accidental” paradigm, stemming from the improper application of FS in an implementation or design.

protocols provides appealing efficiency features—on which we will expand in the remainder of this introduction—but also specific challenges with respect to applying Fiat-Shamir. We then ask:

Research Question. Can the Fiat-Shamir transform yield secure protocols in settings where the verifier does not even access the whole statement?

We are specifically interested in what *crucial* role not reading the whole public input has for the security (or lack thereof) of the Fiat-Shamir transform. We are motivated both by the practical and theoretical appeal of sublinear verification (and its connections to Fiat-Shamir). Sublinear verification is a desirable feature in applications where memory access is relatively expensive (e.g., in a blockchain) or this memory has been delegated in the first place⁵. From a formal standpoint, such access can be modeled by providing an oracle to the statement to the verifier at the beginning of the interaction (we refer to such a verifier as “holographic”). This holographic access is intuitively at the heart of why sublinear protocols have an inherently different relation with Fiat-Shamir. To start with, the verifier does not see the whole input. Moreover, if we want to keep the verifier efficient, we *should not* let it process the whole input to include it for challenge generation (for traditional interactive proofs, not including the whole input is usually sufficient for losing soundness after applying Fiat-Shamir). Our results suggest a nuanced landscape of how Fiat-Shamir specifically breaks for protocols with sublinear verification in general and rational arguments in particular. Before diving more into these questions, we first give a short introduction to the class of proof systems we are interested in.

The Landscape of Sublinear Verification⁶. A first example of protocols with a sublinear verifier is provided by *proofs of proximity* [RVW13]. These extend interactive proofs by shifting the verifier’s task from exact membership to approximate correctness, determining whether an input is close to a valid instance (under a metric such as Hamming distance). In a proof of proximity, the verifier checks whether a given input is “close” to having a certain property, accessing only a portion of its bits. Proofs of proximity apply to useful settings such as checking a codeword of an error-correcting code, and they can be built for (approximately) checking membership in several computational classes (e.g., those decidable by low-depth circuits as in [RVW13]).

Another example of proof schemes in the sublinear setting is the framework of *rational proofs*, introduced by Azar and Micali [AM12]. Unlike purely cryptographic approaches, rational proofs assume the prover is *rational* and *economically incentivized*. To ascertain the output $f(x)$ of a computation, the prover and verifier engage in an interaction after which the verifier assigns a reward based on the transcript. The protocol is designed so that a cheating prover receives a lower expected reward⁷.

Why can we obtain meaningful soundness guarantees in a sublinear setting? It may seem paradoxical that a verifier not reading the whole input can lead to reasonable soundness guarantees. However, these classes of schemes propose different soundness notions that make

⁵ This is not a far-fetched scenario: a client may upload data x to a trusted external memory that later returns the original bits upon request. The verifier queries this memory randomly during the interaction, and the memory is trusted only in terms of its read functionality, not in other ways.

⁶ We will occasionally refer to the “setting of proof systems with sublinear verification” simply as the “sublinear setting” for brevity.

⁷ Here, the verifier is effectively a rewarder; we follow the standard terminology despite this mild imprecision.

sublinear verification meaningful. In proofs of proximity, the key to why sublinearity is possible is that the verifier only checks *distance* to a statement in the language. That is, we are not trying to be persuaded of an exact property holding for the input x ; instead, we are checking a bound on the distance between x and any other input satisfying the property. In rational proofs, sublinearity is also possible because of the weaker security requirement: it is only in *expectation* (over the verifier’s randomness) that the reward is maximized by the honest claim. As long as a reward function can be defined so that, by querying only some bits of the input, this property holds, the protocol is considered secure. Querying only a few bits of the input suffices for this [AM12].

Appealing Features of Rational Proofs. While part of our results will be general—applicable to different kinds of schemes with sublinear verifiers—some of our results will be specific to the setting of rational protocols. We believe rational arguments to be a well-motivated starting point for our main research question, mainly due to their practical appeal and simplicity. Under the assumption of economic incentives, rational protocols offer an efficient replacement for heavier-duty cryptographic proofs in the setting of verifiable computations⁸; the prover in such protocols often incurs no overhead beyond the computation of f itself (proving often involves supplying raw segments of the computation); being simpler, these protocols are easier to implement and reason about. The incentive-flavored nature of rational protocols makes them particularly apt for the setting of smart contracts, especially in their non-interactive version (see also discussion in Appendix E). Finally, because of their relatively simple inner workings, they represent an approachable starting point for the study of Fiat-Shamir in the sublinear regime.

1.1 Our Contributions in a Nutshell

Abstract Framework: “Holographic” Fiat-Shamir and (General) Attacks Against It.

We first introduce a natural analogue of the Fiat-Shamir (FS) paradigm in the sublinear setting, which we denote by hFS (“holographic” FS). Within this model, we formalize a broad class of adversarial strategies, which we refer to as *true-to-false input* (TTF) attacks. This notion is highly general and is compatible with any protocol where the verifier has holographic access to its public input. We show how this type of attack directly yields vulnerabilities for Fiat-Shamir on schemes with sublinear verification. That is, we prove that for any *interactive* (rational or of proximity) scheme Π admitting a TTF adversary, the resulting compiled protocol hFS[Π] will necessarily be insecure. This abstract framework enables the formal development of our subsequent results and is of independent interest.

Complexity of Boolean Functions $\leftrightarrow$ TTF Vulnerability. We show a connection between the vulnerability to TTF attacks and complexity measures of Boolean functions, specifically sensitivity and influence⁹. In particular, we show that for a function f , any rational proof scheme for f becomes vulnerable to TTF attacks based purely on combinatorial properties of f , under a mild assumption on the verifier’s query structure (satisfied by several arguments in the literature). To exemplify our findings, we state an informal corollary of our main result:

⁸ Despite substantial progress in their design (see, e.g., [Tha22b]), cryptographic proofs still have several limitations. They demand significant resources, typically incurring an overhead of three or more orders of magnitude over computing f itself, and, in their most practical instantiations, often rely on strong or poorly scrutinized security assumptions [Tha22a, GW11]

⁹ Sensitivity at an input refers to the number of input bits whose individual flipping changes the function’s output; influence of a bit quantifies the probability that flipping it alters the output, over uniformly sampled inputs.

Corollary (FS fails for impressionable functions: Informal Form of Theorem 5.1). Let Π be a rational argument for a function family $\mathcal{F}$. Then Π becomes insecure under compilation via hFS provided that:

- The verifier’s access to the input x after interaction is fully determined by the transcript¹¹;
- There exists $f \in \mathcal{F}$ such that f is *impressionable*, i.e., for a random x , the output $f(x)$ changes with high probability upon flipping any individual bit of x (Definition 5.4).

Implication: Existing Protocols are Insecure if Compiled with hFS. By applying our abstract framework and our general results on Boolean functions to representative rational arguments in literature, we are then able to show concrete attacks against Fiat-Shamir in the sublinear world. In particular, we show that protocols in [AM13] and [CG15]¹² are insecure when compiled with hFS. We argue this by showing that they: *i*) are susceptible to TTF attacks in their interactive form; *ii*) satisfy the conditions on the verifier’s access pattern that our general attacks require. We prove (*i*) by showing different types of Boolean functions where TTF is possible (respectively among Boolean circuits of logarithmic depth for [CG15] and in threshold circuits of constant depth for [AM13]). Arguing (*ii*) is immediate for [CG15] (because the queried bits are deterministic functions of the transcript), whereas it requires additional observations for the probabilistic query algorithm in [AM13].

TTF Attacks Do Not Require Full Adaptivity. Having shown concrete attacks in the sublinear regime, we turn our attention to studying their essence and what could be preventing them. We study two ways of weakening the adversary: preventing the adversary to select a false claim in a fully adaptive manner; “forcing” the whole input to be used for challenge generation (we expand on this item later in this subsection). By analyzing these two variant scenarios, we find evidence that the nature of TTF attacks seems to be different from that of the usual attacks against weak-FS in traditional interactive proofs (see also § 1.2).

In order to weaken adaptivity, we propose a natural soundness notion for both rational arguments and arguments of proximity, which we call *semi-adaptive*. Its security experiment forces the adversary to declare the non-holographic part of the input (e.g., the function f to evaluate) *before* being able to see the public parameter of the scheme and to interact with the random oracle. It is straightforward to show that even weakening the adversary in this manner does not protect a scheme compiled with hFS against TTF attacks. We discuss other ways of weakening adaptivity and their impact in § 1.2.

A “Strengthened” Fiat-Shamir Variant and Additional Attacks. We then propose a natural strengthening of the syntax and semantic properties of rational arguments¹³, which actually prevents TTF attacks (even in the presence of fully adaptive adversaries). In this model, we let the verifier have access to a hash—a *digest*—of the full input, presumably preprocessed during an offline stage. We then adapt our initial transform, hFS, accordingly to include this

¹¹ This does not require the verifier to be fully deterministic: challenge generation remains randomized, and even if post-query processing involves fresh randomness, the result holds.

¹² These are two early works showing rational proofs with polynomial-time provers for languages within P. These papers are good case study candidates both because they adopt slightly different techniques to build rational proofs (mainly based on scoring functions and algorithmic in flavor, respectively) and representative of the approaches used in most of the follow-up works.

¹³ Note: although we do not formally describe this in our work, a similar variant is possible for arguments of proximity as well.

digest during challenge generation. We denote the resulting transform by digFS (abbreviated to convey “digest” FS).

We show that this variant remains, nevertheless, vulnerable to attacks, albeit of a different kind, at least in some protocols. Specifically, we exhibit such attacks in the $\text{digFS}[\cdot]$ setting AM13 and CG15. This shows that even this strengthened Fiat-Shamir variant does not, in general, suffice to secure rational arguments in the presence of sublinear verifiers.

In the attacks we find, the adversary exploits the challenge distribution to guide the verifier away from the portion of the input that “would reveal” the falsity of the claim¹⁴. Although some challenge paths would reveal the inconsistency, the adversary is always able to find alternative ones where she could cheat undetected. We believe these attacks are of independent interest, and we conjecture they may be generalized in order to attack broader classes of rational arguments compiled with digFS .

1.2 Discussion and Open Questions

On the Role of Adaptivity. We start by discussing the role of adaptivity for TTF attacks. TTF adversaries exploit the fact that the setting is adaptive, but only up to a point: they are able to break both adaptive and semi-adaptive soundness, where the latter requires the adversary to provide the challenger with the non-holographic part of the claim before seeing any protocol-related parameters. We observe that forcing the adversary to be even “less” adaptive in a different way can actually prevent TTF attacks. For example, if it were forced to provide the holographic input before (with or without the non-holographic part), then TTF attacks would not be possible. The input-digest model, in fact, can be seen as a step in this direction. What is important to stress is that it does not force the adversary to pick the holographic input beforehand (the adversary is still adaptive). What it does is simply to prevent the adversary from reusing a transcript for different holographic inputs. (Later in this section, we will remark that there are in fact constructions for which there are no TTF attacks, but that are insecure if the holographic input can be chosen adaptively)

The attacks we show in § 7 in the input-digest model (from now on simply referred to as *input-digest attacks*) show a different behavior. In contrast to TTF attacks, our input-digest attacks do not work in the semi-adaptive setting¹⁵. Later in this section, we will point to this as a lens under which input-digest attacks do not subsume our results on TTF.

More on TTF vs Input-Digest Attacks. Our input-digest attacks in § 7 do not just show that $\text{digFS}[\Pi_{\text{AM13}}]$ and $\text{digFS}[\Pi_{\text{CG15}}]$ are insecure, but also already imply that $\text{hFS}[\Pi_{\text{AM13}}]$ and $\text{hFS}[\Pi_{\text{CG15}}]$ are insecure¹⁶. This observation suggests two natural questions: Doesn’t this suggest that there is something already “vulnerable” about rational arguments unrelated to TTF attacks? If that is the case, are your results on TTF actually providing any useful insight? We believe that the answers to this question are yes and no, respectively. We argue that the TTF attacks are offering insights and providing vulnerabilities that the input-digest attacks are not, but at the same time,

¹⁴ At least at an intuitive level, we find plausible future work might find a connection between these attack strategies and common notions in certificate complexity.

¹⁵ Interestingly, they would work in a non-fully-adaptive setting where the non-holographic input is the only one the adversary is forced to pick before seeing any protocol-related parameters.

¹⁶ It is easy to see, by inspection of our input-digest attacks, that they also apply to the protocols transformed via $\text{hFS}[\cdot]$, at least in the *adaptive* soundness game.

input-digest attacks suggest that there is *also* something else in Π_{AM13} and Π_{CG15} (and potentially in rational argument in general) that makes them vulnerable.

The input-digest attacks have a different “dynamic” than TTF ones. TTF adversaries are comparable to a sleight-of-hand-style con-man (“*Here is an honest transcript for a true claim. Here is the same transcript for a false claim*”). Our input-digest attacks, in contrast, work by brute-forcing a function and transcript for which they would be able to fool the rewarding party (i.e., the verifier). The fact that this brute forcing is possible in polynomial time suggests that the usual soundness properties of rational arguments are potentially amenable to this. What is interesting is that, at least from the face of it, they are also exploiting the sublinearity of the verifier to an extent (see also the discussion on weak-FS in this section). We conjecture it would be possible to find formal general results that shed light on input-digest attacks in rational arguments, and we believe this is an interesting open problem on the specific security limitations of rational arguments in the non-interactive setting. (as we will discuss in the next paragraphs, however, it is not possible to show that all sufficiently expressive non-interactive rational arguments are vulnerable to input-digest attacks in our style)

At the same time, the input-digest attacks are not telling the whole story, and this is the first reason TTF attacks are interesting. In fact, we can easily prevent input-digest attacks by making the adversary slightly less adaptive, as we did in the semi-adaptive model. Here, the adversary would need to provide the function f to target in the soundness game before being able to interact with the random oracle, making our attacks unfeasible (and seemingly unrecoverable, at least with any straightforward tweak). However, in that setting, TTF attacks still work. But TTF attacks remain interesting for one more reason: as we prove, the general fact that hFS is susceptible to TTF adversaries is also true for arguments of proximity. In this work, we show concrete instantiations of TTF for rational arguments alone, but our framework paves the way for future work instantiating that framework in the proximity setting. We find it plausible that our results relating TTF to the complexity of Boolean functions may extend to proximity. At the same time, this will probably require additional care¹⁷ and different bounds/requirements on the size of the safety set and the query algorithm.

We conclude by making an observation on why digFS cannot be meaningfully strengthened further. We defined the input-digest model (and digFS) as one where the verifier has a digest of the holographic input x . We then showed how this is not sufficient by providing relevant attacks. Would these attacks still hold even if the digest included the rest of the public input (function, output, etc.)? The answer is easily yes. In fact, these are all already included in the challenge generation, as these are not holographic parts of the public input. The qualitative leap that digFS (preventing TTF attacks) stems from having the entire holographic input x affecting challenge generation in the form of a digest.

Contrasting weak-FS and hFS. We now discuss the relation between our holographic FS, hFS, and weak-FS. We also contrast TTF attacks to usual attacks against weak-FS.

We start by providing an intuitive framework that positions hFS and its strengthening, digFS, with respect to the common Fiat-Shamir transforms for interactive proofs (we provide a schematic

¹⁷ Here is an intuition on why. In a rational argument, a TTF adversary finds a $\hat{x}$ “compatible” with a fixed transcript τ for x and such that $f(x) \neq f(\hat{x})$. In an argument of proximity, a TTF adversary should be able to find a $\hat{x}$ that is compatible with τ and ϵ -far from a language.

view of this framework in Table 1). In a traditional interactive proof, the standard FS transform is defined by having all of the view of the verifier included in the challenge generation. Although counterintuitive, this is exactly what happens in hFS; all of the view of the verifier up to that point of the interaction is used in the challenge generation (it is just that sublinearity limits the view of the verifier in the first place to only part of the input). The same is not true for weak-FS, which does not include the public input, part of the verifier’s view. Following this framework, digFS is a way to artificially extend challenge generation with elements that will not necessarily be in the view of the verifier during the interaction. In this sense, digFS has no analogue in the realm of traditional interactive proofs.

Above, we argued why *intuitively* hFS and weak-FS are distinct creatures. It is natural to ask whether TTF attacks (which break hFS) are *actually* of a distinct nature than the ones we are already familiar with in the world of traditional interactive proofs. Could it be, for example, that TTF attacks are just the usual attacks against weak-FS in disguise? The answer is no, and for at least two reasons. First, TTF attacks are crucially exploiting the fact that *some* portions of the input will not be read with overwhelming probability, and this is not applicable to interactive proofs. Second, as we discuss in the following paragraphs, there exists a (fairly expressive) non-interactive rational argument that does *not* admit TTF attacks (nor digFS attacks) but that does admit weak-FS-like attacks. This is further evidence that the “vulnerabilities that TTF attacks exploit” are qualitatively different from those exploited by weak-FS attacks.

Positive Results for Rational Arguments? In this work, we showed that the digFS is not sufficient to securely compile two representative rational arguments. Are there sufficiently expressive non-interactive arguments always insecure in the input digest model when compiled with digFS? In the final version of this work, we will provide some partial results in this direction. We show an interactive rational argument Π for the complexity class NC such that:

- Π admits no TTF attacks;
- $\text{digFS}[\Pi]$ is adaptively secure in the input-digest model;
- $\text{hFS}[\Pi]$ is semi-adaptively secure;
- $\text{hFS}[\Pi]$ is not adaptively secure. Moreover—see also first item—the attacks on $\text{hFS}[\Pi]$ are not stemming from TTF strategies, but they resemble more traditional weak-FS attacks.

We are able to obtain this result following the techniques in [GHRV16] and observing that one can compose an already non-interactive rational argument that is not very expressive with a more expressive interactive proof satisfying round-by-round soundness [CCH⁺18]. The resulting protocol can be shown to be secure as described above, but it can be shown to be insecure by an adaptive adversary if the input digest is not included in the challenge generation. We describe these results in more detail in the full version. It remains an interesting open problem to show the existence of an expressive rational argument that is adaptively secure when compiled with hFS.

Comparison with [CJW25]. Here, we highlight the connections between our results and the concurrent and independent work of Chen, Jin, and Wichs [CJW25] (STOC 2025). The focus of their work is (succinct) non-interactive arguments of proximity—or SNAPs—for both deterministic and non-deterministic computations. We will discuss only the first set of results since it is the most relevant to our work. In particular, they show that there cannot be any adaptively-secure SNAP for P where the total communication $p(n)$ and the number of input bits

Setting	Time ($\mathcal{V}$)	FS type	Input to challenge generation vs View($\mathcal{V}$)
Standard arguments	$\Omega(x)$	weak FS	$\subsetneq$
		standard FS	$=$
Holographic arguments	$o(x)$	hFS	$=$
		(Figure 4)	
		digFS	$\supsetneq$
		(Figure 7)	

Table 1: Comparison of Fiat-Shamir transforms in standard arguments and holographic arguments. The last column refers to whether the input given to the hash function used to generate the challenge receives strictly less ($\subsetneq$), the same ($=$), or strictly more ($\supsetneq$) than the input used for running the decision algorithm (or reward algorithm in the case of rational interactive protocols).

queried by the verifier $q(n)$ is such that $p(n) \cdot q(n) = \Omega(n)$ (where the SNAP is guaranteeing ε -proximity, for a sufficiently small $\varepsilon > 0$). Their results are unconditional; therefore, they would also apply in the ROM and to constructions obtained through hFS. They are able to show their result by showing a language in P closely related to secret sharing for which such a SNAP is impossible.

We now discuss the difference between our results and theirs. The results apply to arguments of proximity and, at least not immediately, to rational arguments. We believe it is an interesting open problem to try to apply their techniques to the setting of rational arguments (this is out of the scope of our work since our focus is on Fiat-Shamir). *If* their impossibility results extended to rational arguments *and if* they did extend with similar lower bounds, it is reasonable to ask whether they would subsume some of the insecurity results in our work. We observe that it is plausible such a result would imply the insecurity of $\text{hFS}[\Pi_{\text{CG15}}]$. This is because Π_{CG15} supports NC^1 circuits in which Shamir’s secret sharing can be plausibly expressed. It is unclear, however, whether this would be the case for $\text{hFS}[\Pi_{\text{AM13}}]$: the protocol Π_{AM13} supports (uniform) TC^0 ; we are unaware whether secret sharing (and the additional computation required by the specific language in [CJW25]) would be computable in this complexity class.

Even if their results extended completely to rational arguments, our work offers additional tools compared to their results. Consider, as an example, an interactive holographic scheme Π which was either not very succinct (where $p(n) \cdot q(n) = \omega(n)$) or not expressive enough (limited to subsets of P that do not contain secret sharing). The lower bound in [CJW25] would not be applicable; our framework, on the other hand, might be applicable to show that $\text{hFS}[\Pi]$ would not be secure, provided that Π satisfied the mild conditions in our theorems. Our work already provides concrete results for such analysis in the context of rational arguments, plus it provides initial tools for future work to develop similar results for arguments of proximity. We remark that, in addition to immediate implications for the concrete non-interactive protocols we showed to be insecure, our work offers an investigation on Fiat-Shamir and sublinear verification, which is not in the scope of [CJW25]. We conclude by mentioning that it would be an interesting open problem to study whether there are any essential connections between the observations that the results [CJW25] rely on and our techniques related to complexity metrics of Boolean functions.

1.3 Related Work

The rational proof model was introduced by Azar and Micali [AM12], who developed constant-round protocols for $\#P$ assuming unbounded provers. Their setting was extended in [AM13] to handle threshold computations in constant-depth circuits. Campanelli and Gennaro [CG15, CG17] generalized this work to composable settings with bounded-space and bounded-depth provers. Guo et al. [GHRV14] constructed rational arguments for NC^1 with bounded prover resources and sublinear verifier time; their techniques were later extended to all of P in [GHRV16]. More recent work [CGG24, CG18a] explores variants of rational proofs that incorporate extractability and its interaction with fine-grained cryptographic assumptions.

The broader notion of sublinear-time verification originates in the property testing literature, most notably in the work of Ergün, Kumar, and Rubinfeld [EKR04]. As exact verification became infeasible under limited input access, subsequent work focused on relaxed models. Rothblum, Vadhan, and Wigderson [RVW13] initiated the study of interactive proofs of proximity (IPPs), where the verifier distinguishes inputs that are near from those that are far from a language. This setting has since led to a rich literature on proximity proofs [RVW13, IW14, GGR15, GR17, RR20, GR22, CG18b, GRS23, ABDY24] covering various computational tasks and complexity classes.

The Fiat-Shamir transform itself has received extensive theoretical scrutiny [FS87, DNRS99, Bar01, GK03, BKP18, JKKZ21, HLR21, Kiy22]. These works provide both positive results, often under structural assumptions such as special soundness or witness-uniqueness, and impossibility results revealing inherent limitations in specific contexts. On the applied side, Fiat-Shamir protocols are fragile under various deployment choices [DMWG23], particularly when public inputs are omitted from the challenge derivation, resulting in so-called **weak-FS** instantiations. Further vulnerabilities have been identified in interactive settings [BPW12, HLPT20], where subtle inconsistencies in transcript generation lead to concrete attacks.

Outline

We provide a technical overview in § 2. The reader can find preliminaries on rational arguments in § 3. In § 4 we define a general model of holographic arguments (which encompasses both rational arguments and arguments of proximity) and show Fiat-Shamir transform for them. In the same section we describe our abstract framework of TTF attacks; we also show how TTF adversaries directly make protocols compiled with **hFS** insecure (this result holds for both rational arguments and of arguments of proximity). In § 5 we start populating our framework and show how to derive TTF adversaries from complexity metrics of Boolean functions. These results are specific to rational arguments. We then attack specific rational arguments protocols through the results in earlier sections in § 6. In § 7 we show attacks in the input digest model.

2 Technical Overview

In this overview, we use notation conventional in theoretical computer science. Readers encountering unfamiliar notation should refer to § 3.1.

A simple Fiat-Shamir for holographic inputs. Our initial formulation of the Fiat-Shamir transform in the rational setting follows the natural approach: we instantiate its underlying

heuristic by suitably adapting it (“hash all the public inputs and messages up to the current point of the interaction”). This construction is presented in [Figure 4](#).

The astute reader, however, will observe that this procedure necessarily excludes the full input x from the initial hash context, as x is only partially revealed at the end of the interaction. We proceed to examine why this presents a major issue.

General attacks against holographic verifiers: true-to-false input attacks. We identify a class of potential attacks stemming from our previous observation. Consider a generic holographic scheme (one where the verifier will be limited to reading only a sublinear portion of the input). For the sake of this discussion it is not important whether this is a rational proof, a proof of proximity or any other scheme with specific soundness properties; however, for concreteness in this overview, we will consider claims about function evaluation (which is the case in rational proofs). Now consider an input x , a function f , and the honest (Fiat-Shamir) proof π_{hon} used to assert the output $y_{\text{hon}} = f(x)$. By the definition of rational proofs, π_{hon} assures the “maximum expected reward.” Yet, we can show that this same proof may yield the same reward even for a fraudulent output. Here is how: let $\hat{x}$ be an input that is identical to x except in certain positions specified by the set $\mathcal{I}$, under the condition that $f(\hat{x}) \neq f(x)$. Suppose the positions in $\mathcal{I}$ are not among those the verifier observes. In that case, one can receive the *honest* reward for the *false* claim $y = f(\hat{x})$ (example: let x with parity 1 and let $\hat{x}$ be as x but with one non-queried bit that has been flipped). The reason why this “true-to-false statement” attack is potentially feasible is that the input positions queried by the verifier are determined by the random oracle (RO) invocations and rely solely on public parameters y and f , which remain consistent in both scenarios¹⁸.

True-to-false statement attacks from properties of Boolean functions. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be a Boolean function. That is, f takes an n -bit binary string as input and outputs a single bit. Intuitively, we can think of f as a deterministic rule that assigns a label, either 0 or 1, to each possible input string. To reason about how fragile this output is to small changes in the input, we consider the notion of sensitivity [CDR86, KK04]. Roughly speaking, the sensitivity of f at an input x , denoted $s_f(x)$, counts how many individual bit flips in x would change the output $f(x)$. We define

$$s_f(x) = |\{i \in [n] : f(x) \neq f(x^i)\}|,$$

where x^i denotes the string obtained by flipping the i -th bit of x . The global sensitivity of f , written $s(f)$, is the maximum of $s_f(x)$ over all inputs x , and gives a worst-case measure of how volatile f is.

While sensitivity gives us a local, deterministic view of instability, a more nuanced (and probabilistic) notion is that of influence (see [Juk12, § 2.8] for a thorough introduction to the subject). Loosely speaking, the influence of the i -th coordinate, denoted $\text{Inf}_i(f)$, measures how likely it is that flipping bit i of a random input will change the output; i.e.,

$$\text{Inf}_i(f) = \Pr_x[f(x) \neq f(x^i)].$$

¹⁸ Variants of such attacks are recognized in traditional cryptographic proof settings (see, e.g., [DMWG23]). While astute readers may find such attacks plausible, the implications for verification and rewarding mechanisms are not immediately apparent. The soundness notions in traditional versus rational interactive proofs diverge significantly; the former permits the verifier to access the *entire* input. Accordingly, previous analyses of the Fiat-Shamir approach do not necessarily illuminate our context.

We say that a function is impressionable if every bit has (asymptotically) noticeable influence; in other words, when sampling a random string, changing it by flipping *any* of its bits, we may have a fair chance of changing the function output as well.

Now, recall that in our setting, verifiers often examine only a subset of the input bits. This opens the door for an adversary to tamper with the input in a way that avoids detection. Briefly speaking, a true-to-false adversary tries to produce a valid proof for a slightly different input, an opposite output, while preserving the reward, thereby misleading the verifier.

To model this, we divide the input coordinates into two sets. The query set consists of those bits that the verifier inspects during the protocol. The remaining bits, which the verifier touches with negligible probability, form the so-called safe set. These are the bits that, from the verifier's perspective, are essentially invisible. This setup creates an exploitable gap. An adversary can flip bits in the safe set without being detected. If any of those bits happen to influence the function's output, the adversary may succeed in changing the input while keeping the verifier convinced that the output is correct.

To see how this can go wrong, consider the parity function $\psi(x) = x_1 \oplus x_2 \oplus \dots \oplus x_n$. Intuitively, this function is maximally sensitive: flipping any one bit always flips the output. The sensitivity and influence of every coordinate are both equal to 1. So, if the verifier ignores even a single bit, an adversary can flip that bit and deterministically change the output, while evading detection.

More generally, let $\text{SensitiveSet}(f, x) \subseteq [n]$ denote the set of sensitive coordinates at input x . That is, the coordinates in $\text{SensitiveSet}(f, x)$ are exactly those whose flips affect $f(x)$. If the verifier avoids a subset $S \subseteq [n]$, and some coordinate in S lies in $\text{SensitiveSet}(f, x)$, then the adversary can manipulate $f(x)$ through undetectable changes. Intuitively, the larger the safe set is, the more likely it is to contain some sensitive coordinate, especially if the function has high sensitivity or is impressionable.

This effect becomes statistically exploitable when we consider random inputs and large enough safe sets. For most high-sensitivity or impressionable functions, one can find an input x such that $\text{SensitiveSet}(f, x) \cap S \neq \emptyset$. When that happens, the adversary gains a concrete opportunity to alter the input and flip the output without being caught. Contrast this with monotone Boolean functions like OR and AND, which are only sensitive in very specific cases. For example, $\text{OR}(x)$ is sensitive only when $x := 0^n$, and $\text{AND}(x)$ is sensitive only when $x := 1^n$. These functions resist most bit flips, making them a better choice in this sense (though, as we will see, it is not enough).

To capture broader susceptibility, we say that a Boolean function f is σ -sensitive if there exists some input $x \in \{0, 1\}^n$ for which at least σ coordinates are sensitive. Intuitively, the higher σ is, the more fragile the function becomes at that point. By the pigeonhole principle, if the adversary generates enough large safe sets and samples enough sensitive inputs, then some intersection between a sensitivity set and a safe set must exist.

As long as the adversary confines their manipulations to these sets (and as long as these sets can be efficiently computed), those changes will remain invisible to the verifier. Therefore, if the function is either impressionable or σ -sensitive, and the safe sets are moderately large, the adversary can reliably find a sensitive bit to flip within a safe set. This reveals a fundamental

weakness in the protocol: the combination of structural sensitivity in the function and the verifier’s fixed behavior leads to soundness failures.

The Π_{AM13} and Π_{CG15} Constructions. From the observations above, we are aware of which potential attacks may occur, but it remains unclear whether these attacks manifest in concrete protocols. We show, however, that such attacks are actually possible. Our case studies focus on Π_{AM13} from [AM13], which applies to constant-depth threshold circuits, and Π_{CG15} from [CG15], which targets logarithmic-depth Boolean circuits. Both protocols proceed in a layer-by-layer fashion, but differ in their reward mechanisms: Π_{AM13} utilizes scoring rules¹⁹ (specifically, Brier’s rule [Bri50]), whereas Π_{CG15} uses a specially designed spot-checking method. We show that both protocols are vulnerable to true-to-false statement attacks and are therefore insecure when compiled using the strengthened FS transform discussed above.

Strengthening FS by (Artificially) Including the Full Statement. The attacks we just described rely on a key observation: two *distinct* statements, x and $\hat{x}$, can, loosely speaking, have the prover face the *same* challenges during the proof, and therefore the same set of queried positions. Naturally, this raises the question of whether strengthening the Fiat-Shamir transform could offer a solution. To address this, we consider an augmented model where the verifier retains its usual “spot” access to x , but also takes as input information depending on x in its entirety, a digest²⁰ δ_x . We strengthen the Fiat-Shamir transform by including δ_x in each challenge. As a result, two distinct inputs will now yield distinct digests, effectively preventing the aforementioned attacks. See Figure 7 for the compiler description of our model, which we call the “input-digest model”.

Further Attacks. The goal of this “stronger” FS approach is to provide a litmus test: they make things harder for the adversary assuming *more* from the model (a digest δ_x); if we can show security of rational schemes in this stronger model, then there may be hope to later develop an intermediate Fiat-Shamir approach based on slightly less stringent assumptions. Unfortunately, this is not the case. We can show that both Π_{AM13} and Π_{CG15} are vulnerable to other forms of attack beyond the true-to-false statement manipulations. These new attacks allow an adversary to fix an attempted target input $\tilde{x}$ and then identify a pair (f, y) such that: *i)* $f(\tilde{x}) \neq y$, and *ii)* the reward obtained from proving this false claim is nearly as high as what an honest prover would earn.

CG15 and Attacks in the Input-Digest Model. Let us recall the protocol Π_{CG15} , which operates on functions $f : \{0, 1\}^n \rightarrow \{0, 1\}^n$ computed by a Boolean circuit C of size $|C|$, depth d , and fan-in 2. The circuit C is publicly known to both $\mathcal{P}$ and $\mathcal{V}$, while $\mathcal{P}$ has full access to the input x and $\mathcal{V}^x$ has only oracle access to x . The $\mathcal{P}$ commits to the claimed output $y = C(x)$ along with the purported wire values y_L, y_R at the output gate G , permitting $\mathcal{V}^x$ to check that $G(y_L, y_R) = y$. Verification proceeds by having $\mathcal{V}^x$ randomly select one of the input wires and recursively demand local consistency checks, descending adaptively through the circuit’s topology.

¹⁹ A (proper) scoring rule rewards a forecast provided by an expert and has the key property that any forecast deviating from the true distribution of events (modelled as a probability distribution) will cause the expert to incur a loss. Azar and Micali show how these techniques can be applied to gates in a circuit [AM12, AM13].

²⁰ Intuitively, this can be understood as a compact hash value that the verifier computes once before uploading its input to some memory. See also § 1.

Concretely, $\mathcal{P}$ computes $y_1 = C(x)$ and sends it to $\mathcal{V}^x$, who identifies the root gate g_1 and invokes $\text{loop}(1, g_1, y_1)$. In $\text{loop}(i, g_i, y_i)$, for $1 \leq i \leq d$, $\mathcal{P}$ provides the gate input values z_i^0, z_i^1 for gate g_i , and $\mathcal{V}^x$ checks that $g_i(z_i^0, z_i^1) = y_i$, aborting with zero reward if the gate computation is invalid. When $i = d$, i.e., at the input layer, $\mathcal{V}^x$ verifies that z_i^0, z_i^1 match the corresponding bits of x via queries and pays a reward R if valid. If $i < d$, $\mathcal{V}^x$ samples a random bit $b \in \{0, 1\}$, sends it to $\mathcal{P}$, and recurses into the subcircuit via $\text{loop}(i + 1, g_{i+1}^b, z_i^b)$, where g_{i+1}^b is the child gate producing output z_i^b .

Now, we try and give an intuition for the attack. For illustrative clarity, we instantiate Π_{CG15} with the canonical conjunction function, namely, the n -ary AND circuit realized as a balanced binary tree of fan-in two gates. Both $\mathcal{P}$ and $\mathcal{V}$ have full access to this circuit description, as required by the protocol. The circuit is shown in Figure 1. We then apply our hFS transformation to this instance. The adversary, as will become evident, adaptively mutates the circuit’s structure in response to the Fiat-Shamir challenges, leveraging the deterministic nature of the RO to bias the verifier’s challenge path in its favor.

The attack unfolds by exploiting two core properties. First, the syntactic changes introduced by the adversary are strictly local: the adversary rewires a single AND gate to a OR gate within a semantically inert subtree, one that evaluates to “zero” or “one” regardless of gate type, thus preserving the global output of the circuit on the fixed input. Second, the deterministic interface of the RO allows the adversary to adapt to the Fiat-Shamir challenge, iterating over such local modifications and querying the oracle until a challenge path is obtained that bypasses the modified region and traverses a path aligned with the adversary’s goal. This synthesis of circuit-locality and oracle-adaptivity gives the adversary the ability to construct an accepting proof for a false claim.

We can see, intuitively, that the adversary exploits the structural degrees of freedom in the circuit representation without modifying its semantic behavior on the fixed input $x = \{0^{n/2} \parallel 1^{n/2}\}$. Although the canonical conjunction circuit $C^\wedge$ correctly evaluates $f(x) = 0$, the adversary rewires a single $\wedge$ gate to a $\vee$ gate in a syntactically isolated, semantically inert region, specifically, a zero-valued subtree. This localized perturbation preserves the output on x but perturbs the circuit’s hash digest, thereby inducing a syntactically distinct Fiat-Shamir transcript. Observe Figure 1 to understand that it was $n/2$ gates as options which the adversary could loop over and modify until it obtains a favorable challenge; once it crosses this layer and has successfully made the verifier look the other way, the adversary has nothing to worry about. Each such rewired circuit enables the adversary to extract an independent challenge bit from the RO, thereby generating statistically independent Fiat-Shamir paths while holding both the input and claimed output fixed.

We can see that when the challenge path falls into the favorable domain, namely, avoids the gates where the output is not what the adversary wants, zero in this case, and instead traverses the constructed right subtree (which has the size of $n/4$), the adversary gains full syntactic control. On this path, it can answer every challenge with the pair $(1, 1)$ for the intermediate wires, ensuring every gate check passes without detection. The verifier, unaware of the adversarial manipulation and unable to distinguish the rewired structure from a valid one, accepts the proof and issues a reward for a false claim. Since each attempt succeeds with constant probability and all rewired circuits are functionally correct on x , the adversary requires only $n/2$ trials to succeed with overwhelming probability. This poses a major problem: the Fiat-Shamir transform, even

under the input-digest model, fails to enforce internal consistency and allows provably dishonest proofs to pass verification without incurring any penalty. The scheme $\text{hFS}[\Pi_{\text{CG15}}]$ thus collapses.

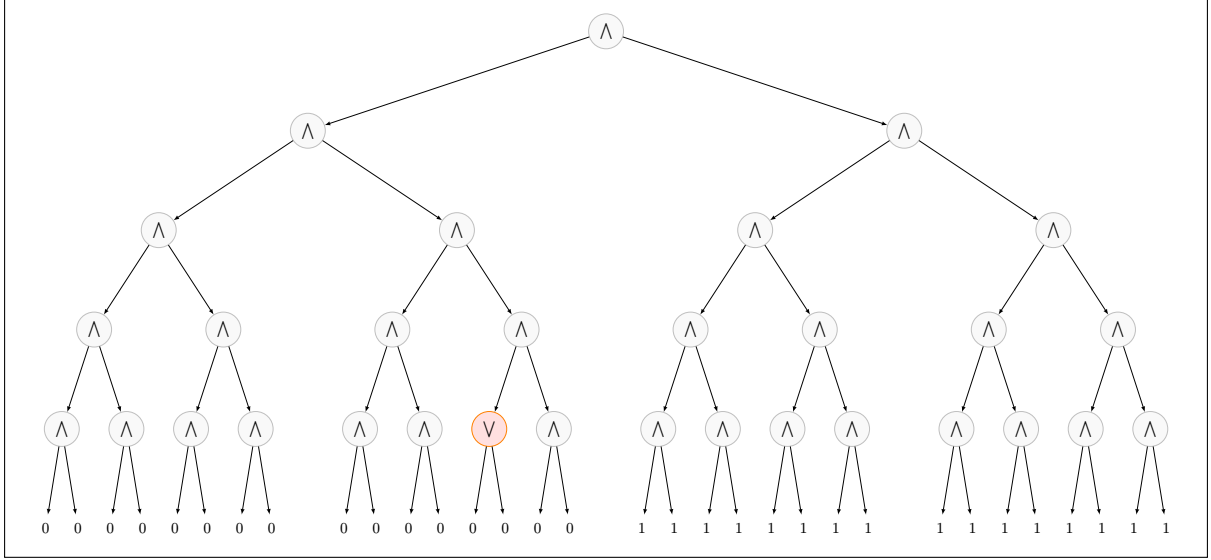


Figure 1: Attack on $\text{hFS}[\Pi_{\text{CG15}}]$

3 Background on Rational Arguments

3.1 Preliminaries and Notation

In this section and throughout the paper, we fix the following notation.

Basic Conventions. The symbol n always refers to a natural number $n \in \mathbb{N}$. A binary string $\vec{v} \in \{0, 1\}^n$ has length $|\vec{v}| = n$, and we write $\vec{v}_i$ for its i -th bit, where $i \in [n] := \{1, \dots, n\}$. The empty string is denoted by ϵ . Concatenation of two strings $x, y \in \{0, 1\}^*$ is written $x \parallel y$. Tuples (x, y) are represented as binary strings via a fixed injective, prefix-free encoding.

Boolean Functions and Circuits. A Boolean function is a map $f : \{0, 1\}^n \rightarrow \{0, 1\}$. A Boolean circuit is a directed acyclic graph built from a finite gate set $\mathcal{G}$ (e.g., $\{\text{AND}, \text{OR}, \text{NOT}\}$), with fan-in at most 2, inputs connected to the bits of x , and a unique output gate. We denote by C_n the set of circuits of size n over $\mathcal{G}$, and by $\llbracket C \rrbracket(x)$ the output of $C \in C_n$ on input $x \in \{0, 1\}^n$. If g_i is a gate of C , then $C[g_i \mapsto t]$ denotes the circuit obtained by changing g_i to type $t \in \mathcal{G}$, leaving all else unchanged. For any input $x = (x_1, \dots, x_n) \in \{0, 1\}^n$ and any coordinate $i \in [n]$, define the bit-flipped variant $x^{(i)} \in \{0, 1\}^n$ by setting $x^{(i)} = (x_1, \dots, x_{i-1}, 1 - x_i, x_{i+1}, \dots, x_n)$.

Hamming Distance and Proximity. For $x, y \in \{0, 1\}^n$, the (relative) Hamming distance is $\Delta(x, y) := \frac{1}{n} |\{i \in [n] : x_i \neq y_i\}|$. For a language $\mathcal{L} \subseteq \{0, 1\}^n$, we define $\Delta(x, \mathcal{L}) := \min_{y \in \mathcal{L}} \Delta(x, y)$. Given a parameter $\varepsilon \in [0, 1]$, we say that x is ε -close to y (or to $\mathcal{L}$) if the corresponding distance is at most ε , and ε -far otherwise.

Difference set.. Given two bitstrings $x, \hat{x} \in \{0, 1\}^n$, their difference set is

$$\Delta\mathcal{S}(x, \hat{x}) := \{i \in [n] : x_i \neq \hat{x}_i\},$$

the set of coordinates where they disagree.

Probability and Indicators. The indicator of a predicate P is $\mathbb{1}_{\{P\}}$, equal to 1 when P is true and 0 otherwise. For a distribution $\mathcal{D}$ over domain D , we write $x \xleftarrow{\$} \mathcal{D}$ for the experiment of sampling x from $\mathcal{D}$. When $\mathcal{D}$ is finite and not otherwise specified, it is taken to be uniform.

Asymptotics. A function $\text{negl}(n) : \mathbb{N} \rightarrow \mathbb{R}^+$ is **negligible** if for every constant $c > 0$ there exists $N \in \mathbb{N}$ such that for all $n > N$, $\text{negl}(n) < n^{-c}$. A function $f : \mathbb{N} \rightarrow \mathbb{R}^+$ is **non-negligible** if there exists a constant $c > 0$ and an infinite set $S \subseteq \mathbb{N}$ such that for all $n \in S$, $f(n) \geq n^{-c}$. A function $p : \mathbb{N} \rightarrow \mathbb{R}^+$ is **polynomially bounded** if there exist constants $c, d > 0$ such that for all $n \in \mathbb{N}$, $p(n) \leq c \cdot n^d$. A function $f : \mathbb{N} \rightarrow \mathbb{R}^+$ is **super-polynomial** if for every constant $d > 0$ there exists $N \in \mathbb{N}$ such that for all $n > N$, $f(n) > n^d$. For asymptotic notation we write $f(n) \in \mathcal{O}(g(n))$ if there exist constants $c > 0$ and N such that $f(n) \leq c \cdot g(n)$ for all $n > N$, and $f(n) \in o(g(n))$ if for every constant $c > 0$ there exists N such that $f(n) < c \cdot g(n)$ for all $n > N$.

Miscellaneous. Unless stated otherwise, logarithms are base 2, i.e., $\log(\cdot) = \log_2(\cdot)$. For $x \in \{0, 1\}^*$, we denote its bit-length by $\ell(x)$. Assignments are written $x := a$ and extend componentwise to tuples, e.g., $(x, (y, z)) := (a, (b, c))$. Parentheses $(\cdot)$ are used as wildcards. Additional notation will be introduced as needed.

3.2 Interactive Rational Arguments

We present the formal model of rational argument systems in the interactive setting. In this model, a prover and a verifier engage in a message-passing protocol, and the prover is economically incentivized to follow an honest strategy via a reward mechanism. The central notion of rationality replaces traditional soundness requirements with the assumption that the prover acts to maximize its expected utility. Thus, under rational behavior, the honest strategy becomes dominant. This setting accommodates both computationally bounded and unbounded provers. In the special case where the prover is unbounded and the setup procedure is trivial (i.e., empty), one recovers the notion of a rational *proof*. Our primary interest, however, is in interactive rational arguments involving computationally bounded provers.

REMARK 3.1. The model we describe in this section is one we introduce. As we show in [Appendix A](#), it is fully equivalent to models presented earlier in the literature. The only substantive difference in our model lies in the presentation of the rational soundness definition and is purely cosmetic. We, subjectively, found a more intuitive definition described in the style of a single cryptographic experiment in contrast to the usual presentation described as properties on gaps in rewards (which the reader can find in [Appendix A](#)).

Definition 3.1 (Rational Argument (Syntax)). An interactive rational argument is formally specified by a tuple of probabilistic polytime (PPT) algorithms $(\text{Setup}, \mathcal{P}, \mathcal{V}, \text{reward})$, with the following behavior:

- $\text{Setup}(1^\lambda) \rightarrow \text{pp}$: a setup procedure that takes a security parameter $\lambda \in \mathbb{N}$ and returns a set of public parameters pp shared by both parties.
- $\langle \mathcal{P}, \mathcal{V} \rangle(\text{pp}, x, f, y) \rightarrow \tau$: a public-coin interactive protocol between $\mathcal{P}$ and $\mathcal{V}$, resulting in a transcript τ , where $\mathcal{P}$ receives (pp, x, f, y) and $\mathcal{V}$ is given only pp and uses uniformly random messages.

- $\text{Query}(\text{pp}, f, y) \rightarrow \mathcal{I}$: outputs a set of coordinates $\mathcal{I} \subseteq [n]$ to be opened in x .
- $\text{reward}(\text{pp}, f, y, \tau, \mathcal{I}, \tilde{x}) \rightarrow R \in \mathbb{R}_{\geq 0}$: a payment function that, given the queried bits $\tilde{x}$, returns a nonnegative reward R based on the computation.

We provide the following convenient shortcut notation and define an algorithm reward as follows.
reward ^{x} (pp, f, y, τ):

- $\mathcal{I} = \text{Query}(\text{pp}, f, y)$
- $\tilde{x} = x_{\mathcal{I}}$
- return $\text{reward}(\text{pp}, f, y, \tau, \mathcal{I}, \tilde{x})$

Rational Completeness. This property asserts that any deviation from the honest strategy does not increase the expected reward by more than a negligible amount. Loosely speaking, rational completeness ensures that an honest prover's expected reward is asymptotically optimal, up to a negligible difference. No efficient prover can significantly outperform the honest strategy, guaranteeing that truthful proofs maximize expected payoff within negligible margins.

Definition 3.2 (Rational Completeness). A rational argument satisfies rational completeness if there exists a negligible function $\text{negl}(\cdot)$ such that for every PPT algorithm $\widehat{\mathcal{P}}$, for all $\lambda \in \mathbb{N}$, for all $x \in \{0, 1\}^*$, for all $f \in \mathcal{F}_\lambda$, and for all $y \in \{0, 1\}^*$, the following holds. Next, let $\text{pp} \leftarrow \text{Setup}(1^\lambda)$, $\hat{\tau} \leftarrow \langle \widehat{\mathcal{P}}(\text{pp}, x, f, y), \mathcal{V} \rangle(\text{pp})$, $\tau \leftarrow \langle \mathcal{P}(\text{pp}, x, f, f(x)), \mathcal{V} \rangle(\text{pp})$. Then,

$$\mathbb{E} [\text{reward}^x(\text{pp}, f, y, \hat{\tau})] - \mathbb{E} [\text{reward}^x(\text{pp}, f, f(x), \tau)] \leq \text{negl}(\lambda).$$

Rational completeness is the decision-theoretic analog of classical completeness. Instead of requiring the honest prover to always convince the verifier, it requires that any rational deviation yield only a negligible advantage in expected reward.

Rational Soundness. This property ensures that an adversary who deviates from the honest strategy and outputs an incorrect value for $f(x)$ suffers a polynomially noticeable drop in expected reward, even when adapting to prior interactions. Intuitively, rational soundness guarantees that any adversary who deviates from honest computation and outputs an incorrect value for $f(x)$ incurs a polynomially noticeable penalty in expected reward, even when allowed to adapt strategies across multiple interactions. This property formalizes the economic disincentive against dishonest behavior by bounding the minimum expected payoff from truthful participation.

Definition 3.3 (Rational Soundness). Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ be a family of function domains. A rational argument satisfies rational soundness for $\mathcal{F}$ if, for every (possibly stateful) PPT adversary $\mathcal{A}$, there exists a polynomial $q(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\mathbb{E} [\text{ExpReward}^{\mathcal{A}}(\lambda)] \geq q(\lambda)^{-1},$$

where the adaptive reward game $\text{ExpReward}^{\mathcal{A}}(\lambda)$ is defined in [Figure 2](#).

Rational soundness quantifies the cost of dishonesty. An adversary cannot gain by reporting an incorrect answer, as doing so results in a strictly lower expected reward by a non-negligible amount.

ExpReward ^{$\mathcal{A}$} (λ):	ExpReward ^{$\mathcal{A}$} _{RO} (λ):
$\text{pp} \leftarrow \text{Setup}(1^\lambda)$ $(f, x, y) \leftarrow \mathcal{A}(\text{pp})$ $\tau_{\mathcal{P}} \leftarrow \langle \mathcal{P}, \mathcal{V} \rangle(\text{pp}, x, f, f(x))$ $R_{\mathcal{P}} \leftarrow \text{reward}^x(\text{pp}, f, f(x), \tau_{\mathcal{P}})$ if $(y \neq f(x) \wedge f \in \mathcal{F}_\lambda)$: $\tau_{\mathcal{A}} \leftarrow \langle \mathcal{A}, \mathcal{V} \rangle(\text{pp}, x, f, y)$ $R_{\mathcal{A}} \leftarrow \text{reward}^x(\text{pp}, f, y, \tau_{\mathcal{A}})$ else : $R_{\mathcal{A}} \leftarrow 0$ return $(R_{\mathcal{P}} - R_{\mathcal{A}})/R_{\mathcal{P}}$	$\text{H} \leftarrow \text{SampleRO}(1^\lambda)$ $\text{pp} \leftarrow \text{Setup}^{\text{H}}(1^\lambda)$ $(f, x, y, \tau_{\mathcal{A}}) \leftarrow \mathcal{A}^{\text{H}}(\text{pp})$ $\tau_{\mathcal{P}} \leftarrow \mathcal{P}^{\text{H}}(\text{pp}, f, x, f(x))$ $R_{\mathcal{P}} \leftarrow \text{reward}^{\text{H}, x}(\text{pp}, f, f(x), \tau_{\mathcal{P}})$ if $(y \neq f(x) \wedge f \in \mathcal{F}_\lambda)$: $R_{\mathcal{A}} \leftarrow \text{reward}^{\text{H}, x}(\text{pp}, f, y, \tau_{\mathcal{A}})$ else : $R_{\mathcal{A}} \leftarrow 0$ return $(R_{\mathcal{P}} - R_{\mathcal{A}})/R_{\mathcal{P}}$

Figure 2: Rational soundness experiments respectively in the interactive setting (left) and in the non-interactive setting in the presence of a random oracle (right).

REMARK 3.2. We require two standard additional conditions for a rational argument to be meaningful: that the reward function is bounded from above by some $\text{poly}(\lambda)$, thereby ensuring that the reward conferred to the prover remains within feasibly realizable bounds; that the verifier runs in sublinear time.

3.3 Non-Interactive Rational Arguments in ROM

We now examine the non-interactive formalization of rational arguments within the Random Oracle Model (ROM) (cf. § 4). In this regime, the interaction between prover and verifier is supplanted via the instantiation of a public random oracle, which acts as a deterministic and globally accessible source of coin flips. The verifier is syntactically inlined into the protocol by means of a reward function, which deterministically computes the prover’s remuneration as a function of the submitted transcript.

SYNTAX 3.1. A non-interactive rational argument in the ROM is defined by a triple $(\text{Setup}, \mathcal{P}, \text{reward})$, with the following components:

- $\text{Setup}^{\text{H}}(1^\lambda) \rightarrow \text{pp}$: a setup procedure with oracle access to a random oracle H , returning public parameters pp on input λ .
- $\mathcal{P}^{\text{H}}(\text{pp}, f, x, y) \rightarrow \pi$: a prover algorithm, also with access to H , that outputs a proof string π attesting that $y = f(x)$.
- $\text{reward}^{\text{H}, x}(\text{pp}, f, y, \pi) \rightarrow R \in \mathbb{R}_{\geq 0}$: a reward²¹ function, with oracle access to both H and x , that computes the reward R based on the claimed output and the submitted proof.

This formulation assumes that the verifier is given query access to the input x . The use of H enforces consistency across all parties and captures Fiat-Shamir-style transformations from interactive protocols to the noninteractive counterparts.

²¹ In contrast to the interactive case, here we assume directly the “assembled” version of the reward function. The separation of query and reward in the interactive case are useful to reason about Fiat-Shamir but less important for the final non-interactive result. Nonetheless, we will occasionally use specifically about the analogues of Query and reward (the latter, without oracle access to the input) in the non-interactive case.

Rational Soundness in ROM. We model adaptive rational soundness for the non-interactive case in the presence of a random oracle.

Definition 3.4 ((Adaptive) Rational Soundness in the Random Oracle Model). Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$. A non-interactive rational argument in the ROM satisfies rational soundness for $\mathcal{F}$ if, for all PPT adversaries $\mathcal{A}$ (possibly non-uniform), there exists a polynomial $q(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\mathbb{E} [\text{ExpReward}_{\text{RO}}^{\mathcal{A}}(\lambda)] \geq q(\lambda)^{-1},$$

where the game $\text{ExpReward}_{\text{RO}}^{\mathcal{A}}(\lambda)$ is defined in Figure 2.

4 Fiat-Shamir Transforms in the Sublinear Setting and TTF Attacks

We build two Fiat-Shamir compilers for protocols with holographic verifiers and show where each one works and where it breaks. We start by defining a **generic holographic argument** interface, Definition 4.1, which cleanly separates interaction, query selection, and verdict computation; this single template covers both rational arguments and arguments of proximity. With that scaffold in place, we present the baseline holographic compiler $\text{hFS}[\cdot]$ in § 4.3 and Figure 3: a direct holographic analogue of classical Fiat-Shamir that preserves sublinear queries. We then isolate the main failure mode, the **true-to-false-input** attack in § 4.4, formalized via input query, safe, and difference sets, and show how it erodes both rational soundness and proximity soundness under Fiat-Shamir, including in the semi-adaptive ROM setting of § 4.6. To counter this, we introduce a stronger compiler $\text{digFS}[\cdot]$ in § 4.7 and Figure 6, which binds challenges with a digest of the entire input yet keeps the verifier holographic and sublinear under explicit assumptions, and we sketch why this design blunts the attack surface.

4.1 Generic Holographic Arguments

We now introduce a unifying abstraction for protocols whose verifier operates in sublinear time and has holographic access to the input. The purpose of this abstraction is to separate the structural flow of such protocols — which often follows a common pattern — from the particular instantiations, such as rational arguments or arguments of proximity. In this way, we obtain a clean interface that highlights the roles of the prover, verifier, and the restricted query access to the input.

The following notion is an abstraction for protocols with a sublinear verifier (and holographic access to their inputs). We abstract away what a “claim” for this argument consists of. We distinguish between a holographic part of the input (x) and the rest of the claim (claim). Our verifier below is a public coin “challenge generator” and is used only in the interactive phase. The algorithms **Query** and **Verdict** encapsulate some of the final steps of the client at the end of the interaction: the former specifies which coordinates to open, and the latter computes the final verdict from the opened positions, the claim, and the transcript.

Definition 4.1 (Generic Holographic Argument). A generic holographic argument is a tuple of PPT algorithms (**Setup**, $\mathcal{P}$, $\mathcal{V}_{\text{chal}}$, **Query**, **Verdict**) where $\mathcal{P}$ and $\mathcal{V}_{\text{chal}}$ are interactive. The syntax is as follows:

- **Setup**(1^λ) $\rightarrow$ **pp**: On input the security parameter λ , output public parameters **pp**.

- $\langle \mathcal{P}(\text{pp}, x, \text{claim}), \mathcal{V}_{\text{chal}}(\text{pp}) \rangle \rightarrow \tau$: On input pp , a string $x \in \{0, 1\}^n$, and a claim²² $\text{claim} : \{0, 1\}^n \rightarrow \{0, 1\}$, produce an interaction transcript τ . We refer to $n := n(\lambda)$ as the “input size”.
- $\text{Query}(\text{pp}, \text{claim}, \tau) \rightarrow \mathcal{I}$: Output a set of coordinates $\mathcal{I} \subseteq [n]$ to be opened in x .
- $\text{Verdict}(\text{pp}, \text{claim}, \tilde{x}, \mathcal{I}, \tau) \rightarrow \text{out}$: On input pp , the claim claim , the queried indices $\mathcal{I}$, the subvector $\tilde{x} := x_{\mathcal{I}}$, and transcript τ , output a verdict $\text{out} \in \{0, 1\}^*$.

For clarity, the execution of a generic holographic argument proceeds as follows.

$$\begin{cases} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \tau \leftarrow \langle \mathcal{P}(\text{pp}, x, \text{claim}), \mathcal{V}_{\text{chal}}(\text{pp}) \rangle \\ \mathcal{I} \leftarrow \text{Query}(\text{pp}, \text{claim}, \tau) \\ \tilde{x} \leftarrow x_{\mathcal{I}} \\ \text{out} \leftarrow \text{Verdict}(\text{pp}, \text{claim}, \tilde{x}, \mathcal{I}, \tau) . \end{cases}$$

REMARK 4.1 (ON ADAPTIVE QUERY ALGORITHMS). Syntactically, the query algorithm has no direct access to x and is therefore non-adaptive. This restriction is without loss of generality for our setting: any adaptive querying strategy can be emulated by requesting the prover’s alleged answers to all potential queries, and then including the actually requested indices in a final query stage. This ensures the query interface remains non-adaptive while retaining full generality.

REMARK 4.2 (RATIONAL ARGUMENTS AS GENERIC HOLOGRAPHIC ARGUMENTS). Rational arguments fit naturally into this abstraction. In this case, the claim claim takes the form (f, y) , where f is a function and y an alleged output. The holographic input x encodes the actual evaluation input to f , and the verifier’s queries correspond to the positions of x that it inspects to validate consistency between the transcript and the claimed output.

REMARK 4.3 (ARGUMENTS OF PROXIMITY AS GENERIC HOLOGRAPHIC ARGUMENTS). Arguments of proximity also instantiate this framework. Here, the claim is that a string x is “close” to a language $\mathcal{L} \subseteq \{0, 1\}^n$. In this setting, Query selects a small set of coordinates of x to test, and Verdict uses those coordinates and the transcript to probabilistically distinguish between the case x is close to $\mathcal{L}$ and the case x is far.

4.2 Input Query Sets and Safe Sets

Given our notion of holographic proofs, we can define a series of notions and properties related to their protocol execution and in particular to where the input is queried. All the definitions in this section are parametrized (often implicitly) by a holographic argument Π . Occasionally, they will also be implicitly parametrized by n , the input size of the related protocol execution.

²² Claims are predicates and they are not necessarily efficiently computable. This is the case, for example, for proofs of proximity where $\text{claim}(x)$ denotes whether x has distance at most ε from a language. The only property we require on claims is that they can be represented through a string of size polynomial in λ . For this reason, we will occasionally refer to *claim strings* when we want to emphasize we are only interested in claims as a syntactic object, which will be the case for most of our definitions.

Definition 4.2 (Input Query Set). For public parameters pp , claim claim , and transcript string τ the input query set is defined as:

$$\text{QuerySet}_{\text{pp}}(\text{claim}, \tau) := \left\{ i \in \{1, \dots, n\} \mid \Pr[i \in \text{Query}(\text{pp}, \text{claim}, \tau)] = 1 - \text{negl}(\lambda) \right\}.$$

The next definition, *safe set* (or, also, *safety set*), intuitively captures the set of indices that are not going to be queried on during the execution of the protocol (and are thus “safe” for the type of adversary who wants to change the input).

Definition 4.3 (Safe Set). Given pp , claim claim , and transcript string τ , the safe set is

$$\text{SafeSet}_{\text{pp}}(\text{claim}, \tau) := \left\{ i \in \{1, \dots, n\} \mid \Pr[i \in \text{Query}(\text{pp}, \text{claim}, \tau)] = \text{negl}(\lambda) \right\}.$$

Definition 4.4 (Safe Set Size). We say that Π has a safe set bounded by $\ell : \mathbb{N} \rightarrow \mathbb{N}$ if for any $\lambda \in \mathbb{N}$, any claim claim , and any input $x \in \{0, 1\}^n$ with $n := n(\lambda)$,

$$\Pr \left[|\mathcal{S}| \geq \ell(n) \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \tau \leftarrow \langle \mathcal{P}(\text{pp}, x, \text{claim}), \mathcal{V}_{\text{chal}}(\text{pp}) \rangle \\ \mathcal{S} \leftarrow \text{SafeSet}_{\text{pp}}(\text{claim}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

The following notion captures a sufficient condition on safe sets that some efficient adversaries may exploit (namely the set should be efficiently computable and non-empty).

Definition 4.5 (Admissible Safety Set). We say that a rational proof Π has an admissible safety set if: *i*) there exists an efficient deterministic algorithm that computes $\text{SafeSet}_{\text{pp}}(\text{claim}, \tau)$ (on inputs pp , claim , and τ ; *ii*) for every $\lambda \in \mathbb{N}$, every claim string claim , and every $x \in \{0, 1\}^n$, with $n := n(\lambda)$:

$$\Pr \left[|\mathcal{S}| \neq \emptyset \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \tau \leftarrow \langle \mathcal{P}(\text{pp}, x, \text{claim}), \mathcal{V}_{\text{chal}}(\text{pp}) \rangle \\ \mathcal{S} \leftarrow \text{SafeSet}_{\text{pp}}(\text{claim}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

We make the following simple observation:

Corollary 4.1. If a holographic argument Π has a deterministic querying algorithm that queries a *strictly* sublinear number of input positions, then Π admits an admissible safety set as by [Definition 4.5](#).

PROOF. The proof is immediate. □

4.3 A Fiat-Shamir Transform for Holographic Protocols (hFS)

At a high level, the holographic Fiat-Shamir transform mirrors the classical one: each verifier challenge is derived by hashing the transcript so far. The difference is that the verifier must remain sublinear, so it only checks a small subset of coordinates in the input. The result is a

non-interactive protocol where the prover reconstructs the interaction using the oracle, and the verifier replays it while restricting attention to the queried positions. This is the compiler $\text{hFS}[\cdot]$ that we formalize in Figure 3.

$\mathcal{P}^H(\text{pp}, x, \text{claim}):$	$\mathcal{V}^{H,x}(\text{pp}, \text{claim}, \pi):$
for $i = 1, \dots, r$ $e_i \leftarrow H(\text{chal}, \text{pp}, \text{claim}, m_1, \dots, m_{i-1})$ $m_i \leftarrow \mathcal{P}(\text{pp}, x, \text{claim}; e_i, m_1, \dots, m_{i-1})$ return $\pi \leftarrow (m_1, \dots, m_r)$	$\pi \leftarrow (m_1, \dots, m_r)$ for $i = 1, \dots, r$ $e_i \leftarrow H(\text{chal}, \text{pp}, \text{claim}, m_1, \dots, m_{i-1})$ $\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$ $\mathcal{I} \leftarrow \text{Query}(\text{pp}, \text{claim}, \tau)$ $\tilde{x} \leftarrow x_{\mathcal{I}}$ $\text{out} \leftarrow \text{Verdict}(\text{pp}, \text{claim}, \tilde{x}, \mathcal{I}, \tau)$ return out

Figure 3: Fiat-Shamir transform $\text{hFS}[\Pi]$ for generic holographic arguments

Definition 4.6 (Fiat-Shamir for Generic Holographic Arguments). Consider a public coin generic holographic argument $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V}_{\text{chal}}, \text{Query}, \text{Verdict})$ with r challenge rounds as in Definition 4.1. In the random-oracle model, define $\text{hFS}[\Pi] = (\text{Setup}, \mathcal{P}^H, \mathcal{V}^{H,x})$ as in Figure 3 with random oracle H ; Query and Verdict are unchanged and are invoked inside the verifier.

For sake of concreteness, we also provide an explicit version of $\text{digFS}[\cdot]$ for the rational argument setting (Figure 4).

$\mathcal{P}^H(\text{pp}, f, x, y):$	$\mathcal{V}^{H,x}(\text{pp}, f, y, \pi):$
for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, f, y, m_1, \dots, m_{i-1})$ $m_i \leftarrow \mathcal{P}(e_i)$ return $\pi \leftarrow (m_1, \dots, m_r)$	$\pi \leftarrow (m_1, \dots, m_r)$ for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, f, y, m_1, \dots, m_{i-1})$ $\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$ $\mathcal{I} \leftarrow \text{Query}(\text{pp}, f, y, \tau)$ $\tilde{x} \leftarrow x_{\mathcal{I}}$ $R \leftarrow \text{reward}(\text{pp}, f, y, \tau, \mathcal{I}, \tilde{x})$ return R

Figure 4: Fiat-Shamir transformation for holographic proofs $\text{hFS}[\Pi]$ for the special case of rational arguments

hFS for Arguments of Proximity For arguments of proximity, the transform works the same way: challenges come from the oracle, the verifier samples only a few coordinates of x , and Verdict checks closeness to the language. The interactive model is given in Definition D.3, the Fiat-Shamir variants in Definitions D.5 and D.6, and the non-interactive version in Definition D.4; a full discussion appears in Appendix D.2.

4.4 True-to-False-Input Attacks in hFS

An interactive sublinear verifier checks only a small (usually uniformly sampled) part of its input. Imagine the adversary knew where the verifier was not going to query its input with high probability for a given transcript. Now, if an adversary changes parts of the input that are unlikely to be checked (that is, in the safe set) the verifier will not detect the change. As a concrete intuition, this implies that, for example, in the case of rational proofs, the adversarial prover will still receive the honest reward despite submitting an incorrect proof.

The following definition captures an adversary that is able to exploit the safe set. In particular, the adversary can produce two inputs x and $\hat{x}$, to which we refer as “true” and “false” input respectively, such that: they have distinct outputs $f(x) \neq f(\hat{x})$ ²³; when the verifier is given access to the *false* input but a transcript from the *true* input, it will not be able to detect this from the position it queries. In order to simplify some of our formal treatment later—when talking about implications for Fiat-Shamir—we model honest transcripts as obtained by the adversary through an oracle. In order to capture the essential intuition of this definition, the reader can simply think of these transcripts as computed locally by the adversary.

Definition 4.7 (True-to-False-Input Adversary). Let Π be a rational argument. We say that Π admits a True-to-False-Input Adversary if there exists a sequence of claims $(\text{claim}_\lambda)_{\lambda \in \mathbb{N}}$ and a PPT adversary $\mathcal{A}$ such that for every $\lambda \in \mathbb{N}$:

$$\Pr \left[\begin{array}{c|c} \text{pp} \leftarrow \text{Setup}(1^\lambda) & \text{claim}_\lambda(\hat{x}) = 0 \wedge \\ L_{\text{tr}} \leftarrow \emptyset & (x, \tau) \in L_{\text{tr}} \wedge \\ \left[(x, \hat{x}, \tau) \leftarrow \mathcal{A}^{O_{\text{HonestTr}}(\cdot)}(\text{pp}) \right] & \Delta\mathcal{S}(x, \hat{x}) \subseteq \text{SafeSet}_{\text{pp}}(\text{claim}_\lambda, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

where the oracle O_{HonestTr} is defined as follows:

$O_{\text{HonestTr}}(x')$:

If $\text{claim}_\lambda(x') = 0$ then return $\perp$
 If $\exists \tau$ s.t. $(x', \tau) \in L_{\text{tr}}$ then return τ
 $\tau \leftarrow \langle \mathcal{P}(\text{pp}, x', \text{claim}_\lambda), \mathcal{V}_{\text{chal}}(\text{pp}) \rangle$
 $L_{\text{tr}} \leftarrow L_{\text{tr}} \cup \{(x', \tau)\}$
return τ

Notice that in the definition above, $(x, \tau) \in L_{\text{tr}}$ immediately implies $\text{claim}_\lambda(x) = 1$. Hence, the claim is true for x but false for $\hat{x}$.

4.5 TTF Adversaries Break Fiat-Shamir

Here we show that True-to-False adversaries immediately yield attacks against hFS in rational arguments and arguments of proximity. The intuition is this: a TTF adversary is one that, intuitively can come up with a false claim $(\text{claim}, \hat{x})$ “compatible” with an honest transcript τ for a true claim (claim, x) . If τ is an interactive transcript, this does not necessarily give much

²³ This example is specific for the case of claims that have to do with function evaluation. See Definition 4.7 for the general case.

power to an adversary (the next interaction might not be one “compatible” with $\hat{x}$). However, in the non-interactive setting, the transcript can be used as a perfectly valid proof string for the false claim. Through the lemmas and theorems in this section, we make this intuition precise. In their statements and proofs, we will use the specific claim semantics of rational arguments and arguments of proximity in order to instantiate the general form in [Definition 4.7](#) (see also [Remark 4.2](#) and [Remark 4.3](#)).

Lemma 4.1 (Rational Arguments and True-to-False Attacks). Let $x, x^* \in \{0, 1\}^n$ be inputs such that $f(x) \neq f(x^*)$ and let pp and τ be two strings. Then

$$\Delta\mathcal{S}(x, x^*) \subseteq \text{SafeSet}_{\text{pp}}(f, f(x), \tau) \implies \left| \mathbb{E} \left[\underset{\text{true claim}}{\text{reward}^x(\text{pp}, \underbrace{f, f(x)}_{\text{true claim}}, \tau)} \right] - \mathbb{E} \left[\underset{\text{false claim}}{\text{reward}^{x^*}(\text{pp}, \underbrace{f, f(x)}_{\text{false claim}}, \tau)} \right] \right| \leq \text{negl}(\lambda).$$

PROOF. For each index i in the difference set $\Delta\mathcal{S}(x, x^*)$, [Definition 4.3](#) implies that

$$\Pr \left[i \in \text{Query}^x(\text{pp}, f, f(x), \tau) \right] \leq \text{negl}(\lambda).$$

That is, the probability that i is queried is negligible.

Define the event $\mathcal{E}$ as the one where, during verification, we query *some* of the positions where x and x^* differ. By applying the union bound, we can bound its probability as follows:

$$\Pr[\mathcal{E}] \leq |\Delta\mathcal{S}(x, x^*)| \cdot \text{negl}(\lambda) \leq \text{negl}(\lambda),$$

since $|\Delta\mathcal{S}(x, x^*)|$ is at most polynomial in λ .

Next, we compare the expected rewards for the true (honest) and false claims. Applying marginalization, we write

$$\begin{aligned} & \left| \mathbb{E} \left[\underset{\text{negl}(\lambda)}{\text{reward}^x(f, f(x), \tau)} \right] - \mathbb{E} \left[\underset{\text{always poly}(\lambda)}{\text{reward}^{x^*}(f, f(x), \tau)} \right] \right| \\ &= \overbrace{\Pr[\mathcal{E}]}^{\text{negl}(\lambda)} \cdot \left| \mathbb{E} \left[\overbrace{\text{reward}^x - \text{reward}^{x^*}}^{\text{always poly}(\lambda)} \mid \mathcal{E} \right] \right| + \Pr[\neg\mathcal{E}] \cdot \left| \mathbb{E} \left[\overbrace{\text{reward}^x - \text{reward}^{x^*}}^0 \mid \neg\mathcal{E} \right] \right| \\ &\leq \text{negl}(\lambda). \end{aligned}$$

Above, we used the previous observations as well as the following fact: conditioned to event $\neg\mathcal{E}$ we have $\text{QuerySet}_{\text{pp}}(f, f(x), \tau) = \text{QuerySet}_{\text{pp}}(f, f(x^*), \tau)$ (the same indices are queried for both inputs). As a result, the input to the final reward function will be the same, and therefore:

$$\left| \mathbb{E} \left[\text{reward}^x - \text{reward}^{x^*} \mid \neg\mathcal{E} \right] \right| = 0.$$

This concludes the proof. $\square$

Lemma 4.2 (Arguments of Proximity and True-To-False Attacks). Let $\mathcal{L} \subseteq \{0, 1\}^*$ and Π be an interactive argument of proximity with soundness error $\varepsilon : \mathbb{N} \rightarrow [0, 1]$. Fix $\lambda \in \mathbb{N}$ and $n \in \mathbb{N}$. Let $x, \hat{x} \in \{0, 1\}^n$ with $x \neq \hat{x}$, $x \in \mathcal{L}$, and $\Delta(\hat{x}, \mathcal{L}) > \varepsilon(n)$. Let pp be a string and let τ be the transcript. If

$$\Delta\mathcal{S}(x, \hat{x}) \subseteq \text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau),$$

then

$$\left| \Pr[\text{Decision}^x(\text{pp}, \mathcal{L}, \tau) = 1] - \Pr[\text{Decision}^{\hat{x}}(\text{pp}, \mathcal{L}, \tau) = 1] \right| \leq \text{negl}(\lambda).$$

PROOF (SKETCH). The argument is similar to [Lemma 4.1](#). By hypothesis, the difference set $\Delta\mathcal{S}(x, \hat{x})$ lies entirely inside the safe set. Thus, as shown in [Lemma 4.1](#), the verifier queries the same coordinates on x and $\hat{x}$ except with negligible probability. Therefore, the verifier's acceptance probabilities satisfy the inequality stated in [Lemma 4.2](#). $\square$

This lemma extends the reasoning of [Lemma 4.1](#) to proximity arguments.

Theorem 4.1 (TTF Adversaries Break Fiat-Shamir (Rational Arguments)). Let Π be a secure interactive rational argument. If Π admits a true-to-false-input adversary, then its Fiat-Shamir transformation $\text{hFS}[\Pi]$ is insecure.

PROOF. Let $\mathcal{A}_{TTF}$ be a true-to false adversary for Π , as in [Definition 4.7](#). Recall that a claim in the rational setting is of the form $\text{claim} = (f, y)$ and $\text{claim}(x) = 1 \leftrightarrow f(x) = y$. By the definition of TTF adversary there exists $(\text{claim}_\lambda)_{\lambda \in \mathbb{N}}$ exploitable by $\mathcal{A}_{TTF}$. Below we denote by (f_λ, y_λ) the function–output pair for parameter λ corresponding to claim_λ . Let us build an adversary $\mathcal{A}$ against non-interactive rational soundness ([Definition 3.4](#)) as follows:

$\begin{array}{l} \mathcal{A}^H(\text{pp}): \\ \hline L_{\text{tr}} \leftarrow \emptyset \\ (x, \hat{x}, \tau) \leftarrow \mathcal{A}_{TTF}^{O'_{\text{HonestTr}}(\cdot)}(\text{pp}) \\ \text{return } (f_\lambda, \hat{x}, y_\lambda) \end{array}$	$\begin{array}{l} O'_{\text{HonestTr}}(x'): \\ \hline \text{If } f_\lambda(x') \neq y_\lambda \text{ then return } \perp \\ \text{If } \exists \tau \text{ s.t. } (x', \tau) \in L_{\text{tr}} \text{ then return } \tau \\ \tau \leftarrow \mathcal{P}^H(\text{pp}, f_\lambda, x', y_\lambda) \\ L_{\text{tr}} \leftarrow L_{\text{tr}} \cup \{(x', \tau)\} \\ \text{return } \tau \end{array}$
---	--

We can observe that the oracle O'_{HonestTr} behaves identically to the oracle O_{HonestTr} in [Definition 4.7](#). As a consequence, with overwhelming probability, the tuple $(x, \hat{x}, \tau)$ output by $\mathcal{A}$ satisfies the hypothesis of [Lemma 4.1](#). Hence, the resulting expected reward for the false claim is is close in expectation to that for the honest one:

$$\left| \mathbb{E}[\text{reward}^x(\dots)] - \mathbb{E}[\text{reward}^{\hat{x}}(\dots)] \right| \leq \text{negl}(\lambda).$$

This directly contradicts the rational soundness of $\text{hFS}[\Pi]$. $\square$

Sufficient conditions for the existence of such adversaries are discussed in [§ 5](#).

Theorem 4.2 (TTF Adversaries Break Fiat-Shamir (Proximity Arguments)). Let Π be a secure interactive argument of proximity for $\mathcal{L}$. If Π admits a true-to-false-input adversary as in [Definition D.10](#), then its Fiat-Shamir compilation $\text{hFS}[\Pi]$ is insecure.

PROOF (SKETCH). This is a straightforward adaptation of the proof in [Theorem 4.1](#). $\square$

4.6 Semi-Adaptive Soundness and its Vulnerability to TTF

Here we define a notion of soundness for non-interactive rational arguments where the adversary is constrained to providing the function f in its target claim *before* being able to interact with

the random oracle and seeing the parameters of the scheme. For simplicity we define it only for the rational setting.

Definition 4.8 ((Semi-Adaptive) Rational Soundness in the Random Oracle Model).

Let $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$. A non-interactive rational argument in the ROM satisfies semi-adaptive rational soundness for $\mathcal{F}$ if, for all pairs of PPT adversaries $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$ (possibly non-uniform), there exists a polynomial $q(\cdot)$ such that for all $\lambda \in \mathbb{N}$,

$$\mathbb{E} \left[\text{ExpRewardSemiAdapt}_{\text{RO}}^{\mathcal{A}}(\lambda) \right] \geq q(\lambda)^{-1},$$

where the game $\text{ExpRewardSemiAdapt}_{\text{RO}}^{\mathcal{A}}(\lambda)$ is defined in [Figure 2](#).

ExpRewardSemiAdaptRO ^{$\mathcal{A}$} (λ):

$(f, \text{st}) \leftarrow \mathcal{A}_1(1^\lambda)$
 $H \leftarrow \text{SampleRO}(1^\lambda)$
 $\text{pp} \leftarrow \text{Setup}^H(1^\lambda)$
 $(x, y, \tau_{\mathcal{A}}) \leftarrow \mathcal{A}_2^H(\text{pp}, \text{st})$
 $\tau_{\mathcal{P}} \leftarrow \mathcal{P}^H(\text{pp}, f, x, f(x))$
 $R_{\mathcal{P}} \leftarrow \text{reward}^{H,x}(\text{pp}, f, f(x), \tau_{\mathcal{P}})$
if $(y \neq f(x) \wedge f \in \mathcal{F}_\lambda)$:
 $R_{\mathcal{A}} \leftarrow \text{reward}^{H,x}(\text{pp}, f, y, \tau_{\mathcal{A}})$
else :
 $R_{\mathcal{A}} \leftarrow 0$
return $(R_{\mathcal{P}} - R_{\mathcal{A}})/R_{\mathcal{P}}$

Figure 5: Semi-Adaptive Rational soundness experiment and in the non-interactive setting in the presence of a random oracle.

Theorem 4.3 (TTF Adversaries Break Fiat-Shamir (Semi-Adaptive Rational Soundness)). Let Π be a secure interactive rational argument. If Π admits a true-to-false-input adversary, then its Fiat-Shamir transformation $\text{hFS}[\Pi]$ does not satisfy [Definition 4.8](#).

PROOF. The proof is identical to that of [Theorem 4.1](#). It suffices to additionally observe that the syntax of a TTF adversary suffices to provide an attack in the semi-adaptive soundness experiment ([Definition 4.8](#)). The function provided by $\mathcal{A}_1$ is simply the function $f \in \mathcal{F}_\lambda$ the TTF adversary exploits as by [Definition 4.7](#). $\square$

REMARK 4.4. While we do not formalize it in this work, it is possible to define an analogue of [Definition 4.8](#) for arguments of proximity. An analogue of [Theorem 4.3](#) also holds in the proximity setting.

4.7 A Stronger Fiat-Shamir Compiler with “Input Digests”

We now introduce a variant of the holographic Fiat-Shamir transform that is not susceptible to the TTF attacks described in this section. Here, the verifier’s challenges are generated using the

entire input string x through a digest dig . We let $\text{dig} := H(x)$, where H is a random oracle. Since the computation of $H(x)$ requires full access to x , we assume that dig is explicitly supplied to the verifier as an auxiliary input. This allows us to keep the verifier sublinear²⁴. This transformation is described formally in Figure 7; a complete formalization of the corresponding soundness model is deferred to Appendix B.

$\mathcal{P}^H(\text{pp}, x, \text{claim}):$	$\mathcal{V}^{H,x}(\text{pp}, \text{claim}, \pi):$
$\text{dig} \leftarrow H(x)$	$\pi \leftarrow (m_1, \dots, m_r)$
for $i = 1, \dots, r$	$\text{dig} \leftarrow H(x)$
$e_i \leftarrow H(\text{chal}, \text{pp}, \text{claim}, \text{dig}, m_1, \dots, m_{i-1})$	for $i = 1, \dots, r$
$m_i \leftarrow \mathcal{P}(\text{pp}, x, \text{claim}; e_i, m_1, \dots, m_{i-1})$	$e_i \leftarrow H(\text{chal}, \text{pp}, \text{claim}, \text{dig}, m_1, \dots, m_{i-1})$
return $\pi \leftarrow (m_1, \dots, m_r)$	$\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$
	$\mathcal{I} \leftarrow \text{Query}(\text{pp}, \text{claim}, \tau)$
	$\tilde{x} \leftarrow x_{\mathcal{I}}$
	$\text{out} \leftarrow \text{Verdict}(\text{pp}, \text{claim}, \text{dig}, \tilde{x}, \mathcal{I}, \tau)$
	return out

Figure 6: Input-digest Fiat-Shamir transform $\text{digFS}[\Pi]$ for generic holographic arguments. The differences from $\text{hFS}[\Pi]$ are highlighted in purple.

Definition 4.9 (“Input-Digest” Fiat-Shamir Transform for Generic Holographic Arguments). Given a generic holographic argument Π , the associated non-interactive compiled argument is denoted by $\text{digFS}[\Pi]$, where the compiler $\text{digFS}[\cdot]$ is as in Figure 6.

For sake of concreteness, we also provide an explicit version of $\text{digFS}[\cdot]$ for the rational argument setting (Figure 7).

5 Generic TTF Attacks from Boolean Function Properties

In this section, we introduce basic notions about Boolean functions that we use throughout the paper. We define sensitivity and influence Definitions 5.2 and 5.3, which measure how output values react to flips of input bits. We then describe impressionable function families Definition 5.4, where every input bit has noticeable influence, and formalize the sensitive set of a function at a given input Definition 5.1. These notions give us quantitative control over how functions respond to local changes. In later sections, we use them to build adversaries, prove the existence of true-to-false attacks, and show the insecurity of protocols.

REMARK 5.1 (ON THE GENERALITY OF THE RESULTS IN THIS SECTION). While we state the formal results in this section (Theorems 5.1 and 5.2) for the case of rational proofs, we stress

²⁴ At least in principle, the availability of cryptographic machinery such as RAM SNARGs (cf. [KLVW23]) enables the construction of hash verification procedures that are genuinely sublinear in the input length. More concretely, the prover may compress the input by computing a digest via a somewhere extractable statistically binding hash function, and either (i) provide a proof attesting to the correct computation of the digest, or (ii) rely on the fact that any computationally unbounded third party can verify the digest’s correctness by re-evaluating the hash function. In this sense, the computation of the digest is transparent, as its correctness can be externally validated without reliance on interaction.

$\mathcal{P}^H(\text{pp}, f, x, y):$	$\mathcal{V}^{H,x}(\text{pp}, f, y, \pi):$
$\text{dig} \leftarrow H(x)$	$\pi \leftarrow (m_1, \dots, m_r)$
for $i = 1, \dots, r$	$\text{dig} \leftarrow H(x)$
$e_i \leftarrow H(\text{pp}, f, \text{dig}, y, m_1, \dots, m_{i-1})$	for $i = 1, \dots, r$
$m_i \leftarrow \mathcal{P}(e_i)$	$e_i \leftarrow H(\text{pp}, f, \text{dig}, y, m_1, \dots, m_{i-1})$
return $\pi \leftarrow (m_1, \dots, m_r)$	$\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$
	$I \leftarrow \text{Query}(\tau)$
	$\tilde{x} \leftarrow x_I$
	$R \leftarrow \text{reward}(\text{pp}, f, \text{dig}, y, \tau, \tilde{x})$
	return R

Figure 7: Input-Digest Fiat-Shamir transformation $\text{digFS}[\Pi]$ for rational arguments. In purple we emphasize differences with $\text{hFS}[\Pi]$ (described in Figure 4).

that they are not specific to rational proofs alone. In fact, they immediately extend to any holographic scheme (Definition 4.1) where the winning condition of the adversary is of the form “ $y \neq f(x) \wedge [\text{some condition on the verdict}]$ ”. To the best of our knowledge, rational proofs are currently the only type of holographic proof with this property in literature²⁵. Our techniques are not immediately compatible with the setting of proofs of proximity. This is because their winning condition is defined in terms of the *distance* of the input provided to the verifier to a language.

5.1 Preliminaries on Boolean Functions

Here, we define standard useful properties on Boolean functions, such as sensitivity and influence. We then provide a few definitions to classify Boolean functions according to these properties and derive some simple facts that will be useful in our proofs. We start by defining the notion of sensitive set of a function f on input x , which corresponds to the coordinates at which f is sensitive at x .

Definition 5.1 (Sensitive Set). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and let $x \in \{0, 1\}^n$. The sensitive set of f at x , denoted $\text{SensitiveSet}(f, x)$, is defined as

$$\text{SensitiveSet}(f, x) := \{i \in [n] : f(x^{(i)}) \neq f(x)\}.$$

Definition 5.2 (Sensitivity). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be a Boolean function. The sensitivity of f at an input $x \in \{0, 1\}^n$, denoted $s_f(x)$, is defined as the number of coordinates $i \in [n]$ for which flipping the i -th bit of x alters the value of f , i.e.,

$$s_f(x) := |\text{SensitiveSet}(f, x)|.$$

The global sensitivity of f is given by

$$s(f) := \max_{x \in \{0, 1\}^n} s_f(x).$$

²⁵ One of the authors, however, has recently worked on a different model with this property that would fit this template (the work is still unavailable as a preprint at the time of this writing); thus, rational proofs are not unique in this sense.

If a function f is such that $s_f(x) \geq \sigma$, for $\sigma \in \mathbb{N}$, we say the function is σ -sensitive.

Definition 5.3 (Influence). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be a Boolean function and fix a coordinate $i \in [n]$. The influence of the i -th variable on f , denoted $\text{Inf}_i(f)$, is defined as the probability over uniformly random inputs $x \in \{0, 1\}^n$ that flipping the i -th bit changes the output of f , i.e.,

$$\text{Inf}_i(f) := \Pr_x[f(x) \neq f(x^{(i)})].$$

A family of Boolean functions is said to be *impressionable* if the influence of each coordinate is noticeable.

Definition 5.4 (Impressionable Functions). Let $\mathcal{F} = (f_\lambda)_{\lambda \in \mathbb{N}}$ be a family of Boolean functions where each f_λ has domain $\{0, 1\}^{n(\lambda)}$ where $n(\cdot)$ is a fixed polynomially bounded function. The family $\mathcal{F}$ is said to be impressionable if there exists a constant $c > 0$ such that for every $\lambda \in \mathbb{N}$ and every $i \in [n]$, with $n := n(\lambda)$ we have $\text{Inf}_i(f_\lambda) \geq n(\lambda)^{-c}$.

We will occasionally abuse notation to simplify the statements of our theorems: when referring to a Boolean function f as impressionable, we mean that it belongs to some impressionable sequence $\mathcal{F} = (f_\lambda)_{\lambda \in \mathbb{N}}$ in the sense of this definition. Whenever we write n to refer to a concrete input size, it is implicitly assumed $n := n(\lambda)$.

Lemma 5.1. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be impressionable in the sense of [Definition 5.4](#), and let $S \subseteq [n]$ be a nonempty set. Then

$$\Pr_x[S \cap \text{SensitiveSet}(f, x) \neq \emptyset] \geq n^{-c}.$$

PROOF. Fix an arbitrary coordinate $j \in S$. By definition of impressionability [Definition 5.4](#), we have

$$\text{Inf}_j(f) = \Pr_x[j \in \text{SensitiveSet}(f, x)] \geq n^{-c}.$$

Since the event $j \in \text{SensitiveSet}(f, x)$ implies that $S \cap \text{SensitiveSet}(f, x) \neq \emptyset$, the desired inequality follows. $\square$

Lemma 5.2 (Sensitive Set Coverage). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be an impressionable Boolean function. Then there exists a constant $k > 0$ such that for $t = n^k$ and for any collection of nonempty subsets $S_1, \dots, S_t \subseteq [n]$, the following holds:

$$\Pr_{x_1, \dots, x_t} [\exists i \in [t] \mid S_i \cap \text{SensitiveSet}(f, x_i) \neq \emptyset] \geq 1 - \text{negl}(n).$$

PROOF. Set $p := n^{-c}$. From [Lemma 5.1](#), for each $i \in [t]$ we have

$$\Pr[S_i \cap \text{SensitiveSet}(f, x_i) \neq \emptyset] \geq p.$$

Let $\mathcal{E}_i$ denote the event that S_i intersects $\text{SensitiveSet}(f, x_i)$. These events are mutually independent across i , since the x_i are independently sampled. Hence, the probability that none of the intersections occur is bounded by

$$\Pr \left[\bigcap_{i=1}^t \mathcal{E}_i^c \right] = \prod_{i=1}^t \Pr[\mathcal{E}_i^c] \leq (1 - p)^t \leq e^{-pt}.$$

Choosing $t = \lceil n^{c+1} \rceil$ ensures that $pt \geq n$, yielding a failure probability of at most e^{-n} . The desired bound follows. $\square$

5.2 Attacks from Impressionable Functions

Theorem 5.1 (True-to-False Attack for Families of Impressionable Functions). Let Π be a rational argument protocol for an impressionable Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ and with an admissible safety set. Then Π is insecure against a true-to-false adversary.

PROOF. We construct a true-to-false adversary $\mathcal{A}$ as follows (k is an appropriately chosen constant):

$$\begin{array}{l} \mathcal{A}^{\mathcal{O}_{\text{HonestTr}}(\cdot)}(\text{pp}): \\ \hline \text{For } t = 1, \dots, n^k : \\ \quad \text{Sample } x \xleftarrow{\$} \{0, 1\}^n \\ \quad \tau \leftarrow \mathcal{O}_{\text{HonestTr}}(x) \\ \quad \mathcal{S} \leftarrow \text{SafeSet}_{\text{pp}}(f, f(x), \tau) \\ \quad \text{if } \mathcal{S} \cap \text{SensitiveSet}(f, x) \neq \emptyset : \\ \quad \quad \text{Let } j^* \in \mathcal{S} \cap \text{SensitiveSet}(f, x) \\ \quad \quad \text{return } (x, x^{(j^*)}, \tau) \end{array}$$

By Lemma 5.2, one of the t sampled inputs flips succeeds with overwhelming probability, yielding $j^* \in \mathcal{S} \cap \text{SensitiveSet}(f, x)$, which implies $f(x^{(j^*)}) \neq f(x)$ and $\Delta\mathcal{S}(x, x^{(j^*)}) \subseteq \text{SensitiveSet}(f, x)$. This concludes the proof. $\square$

Corollary 5.1 (Insecurity of Protocols for Impressionable Functions). Let $(f)_{\lambda \in \mathbb{N}}$ be a family of impressionable Boolean functions and Π a rational argument protocol for this family with admissible safety sets. Then $\text{hFS}[\Pi]$ is insecure.

PROOF. This follows immediately from Theorem 5.1. $\square$

5.3 Attacks from σ -Sensitivity

Lemma 5.3 (Safety Set Collision). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be σ -sensitive and suppose $\mathcal{S} \subseteq [n]$ satisfies $|\mathcal{S}| \geq n - \sigma + 1$. Then there exists $x \in \{0, 1\}^n$ such that $s_f(x) \geq \sigma$ and $\mathcal{S} \cap \text{SensitiveSet}(f, x) \neq \emptyset$.

PROOF. By σ -sensitivity, there exists $\hat{x} \in \{0, 1\}^n$ such that $|\text{SensitiveSet}(f, \hat{x})| \geq \sigma$. Let $\mathcal{S}^* := [n] \setminus \mathcal{S}$, so $|\mathcal{S}^*| \leq \sigma - 1$. Since $|\text{SensitiveSet}(f, \hat{x})| > |\mathcal{S}^*|$, there exists $j \in \text{SensitiveSet}(f, \hat{x})$ with $j \notin \mathcal{S}^*$. Thus, $j \in \mathcal{S} \cap \text{SensitiveSet}(f, \hat{x})$. $\square$

Theorem 5.2 (True-to-False Attack from σ -Sensitivity). Let Π be a rational argument protocol for a Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ that is σ -sensitive. If Π admits an admissible safety set bounded by $\ell \geq n - \sigma + 1$, then Π admits a true-to-false adversary.

PROOF. The adversary is described in Figure 8. By invoking Lemma 5.3 and the definition of sensitive set we can conclude it is a TTF adversary. $\square$

$\mathcal{A}_{\text{HonestTr}}^{(\cdot)}(\text{pp}):$ Let $x \in \{0, 1\}^n$ s.t. $s_f(x) \geq \sigma$ $\tau \leftarrow \mathcal{O}_{\text{HonestTr}}(x)$ $\mathcal{S} \leftarrow \text{SafeSet}_{\text{pp}}(f, f(x), \tau)$ Let $j^* \in \mathcal{S} \cap \text{SensitiveSet}(f, x)$ return $(x, x^{(j^*)}, \tau)$

Figure 8: TTF adversary for [Theorem 5.2](#). Here, $x \in \{0, 1\}^n$ and j^* is any index in the nonempty intersection $\mathcal{S} \cap \text{SensitiveSet}(\phi, x)$.

Corollary 5.2. Let Π be a rational argument protocol for $f : \{0, 1\}^n \rightarrow \{0, 1\}$ admitting an admissible safety set bounded by ℓ . If f is σ -sensitive and $\ell \geq n - \sigma + 1$, then $\text{hFS}[\Pi]$ is insecure.

PROOF. The claim follows directly from [Theorem 5.2](#). □

REMARK 5.2 (ON THE DIFFERENCE COROLLARIES 5.1 AND 5.2). [Corollary 5.1](#) applies to impressionable functions and has essentially no requirements on the safety set. [Corollary 5.2](#), on the other hand requires a bound on the size of the safety set but it relies on sensitivity of the target function—a property easier to reason about than impressionability.

6 Concrete TTF Attacks Against Rational Arguments

6.1 Background on Our Case Studies

Azar-Micali Rational Interactive Protocol [[AM13](#)]. The protocol is described in [Figure 9](#). We now provide an intuitive interpretation of the protocol starting from the special case of a single threshold gate. Consider the evaluation of a threshold function $f_t : \{0, 1\}^n \rightarrow \{0, 1\}$ defined by

$$f_t(x_1, \dots, x_n) = \begin{cases} 1, & \text{if } \sum_{i=1}^n x_i \geq t, \\ 0, & \text{otherwise,} \end{cases}$$

where $n \in \mathbb{N}$ denotes the number of Boolean input bits, and $t \in [n]$ is the threshold parameter.

The prover announces a claimed value $\tilde{m} \in [n]$, which is purported to be the exact Hamming weight of the input vector $\vec{x} = (x_1, \dots, x_n) \in \{0, 1\}^n$, that is, $\tilde{m} = \sum_{i=1}^n x_i$. Based on this declaration, the verifier computes the gate output as $\mathbb{1}_{\{\tilde{m} \geq t\}}$. Subsequently, the verifier samples an index $i \in [n]$ uniformly at random and observes the corresponding input bit $b := x_i$. This sampled bit serves as a statistical test of the prover's claimed value. Defining $\tilde{p} := \frac{\tilde{m}}{n}$ as the purported marginal probability that a uniformly chosen input bit is equal to 1, the verifier computes the reward according to Brier's Scoring Rule (BSR) [[Bri50](#)]:

$$\text{BSR}(\tilde{p}, b) = \begin{cases} 2\tilde{p}(2 - \tilde{p}), & \text{if } b = 1, \\ 2(1 - \tilde{p}^2), & \text{if } b = 0. \end{cases}$$

Denote by $p := m/n$ the true marginal probability that an input bit equals 1. The expected reward for the prover, taken over the random choice of i , is therefore

$$\mathbb{E}[\text{reward}] = p \cdot \text{BSR}(\tilde{p}, 1) + (1 - p) \cdot \text{BSR}(\tilde{p}, 0).$$

By direct calculation, this expectation is maximized if and only if $\tilde{p} = p$. Any deviation from the true value $\tilde{p} \neq p$ incurs an expected penalty bounded below by $2(p - \tilde{p})^2$, implying that any one-bit misreport yields a minimal expected loss of at least $2/n^2$. Now, consider a dishonest prover strategy $\widehat{\mathcal{P}}$ which deviates from truthful reporting with probability $\epsilon_{\widehat{\mathcal{P}}}$. The expected loss $\delta_{\widehat{\mathcal{P}}}$ induced by this deviation satisfies $\delta_{\widehat{\mathcal{P}}} > \frac{2\epsilon_{\widehat{\mathcal{P}}}}{n^2}$. Hence, the protocol ensures rationality: no prover can increase expected reward by deviating from truthful reporting.

The protocol proceeds by iterating the procedure above level by level and aggregating the rewards of the prover at each level. Its soundness is ensured by appropriately *discounting* the reward of the prover at level j by a scaling factor δ_j . The final result is a protocol that is secure for circuit families with threshold gates of constant depth.

Definition 6.1 (T -Uniformity). We say that a circuit family $\{C_n\}_{n \in \mathbb{N}}$ is T -uniform if there exists a deterministic algorithm that, on input $(1^n, g, h, i)$, decides whether h is the i -th input to gate g in circuit C_n , and on input $(1^n, g, t)$, decides whether gate g in C_n is of type t , both in time $T(n)$. That is, the union of the following two languages:

$$\{(1^n, g, h, i) : h \text{ is the } i\text{-th input to gate } g \text{ in } C_n\} \cup \{(1^n, g, t) : g \text{ is a gate of type } t \text{ in } C_n\}$$

is decidable in time $T(n)$.

Theorem 6.1 ([AM13]). The protocol Π_{AM13} described in Figure 9 constitutes a secure interactive rational argument for T -uniform circuit families with threshold gates of $\mathcal{O}(1)$ depth. It runs in a constant number of rounds, has verification complexity $\mathcal{O}(\log n \log \log n + T(n))$, and requires only a constant number of queries to the input vector $\vec{x}$.

REMARK 6.1. We will later use the following facts from [AM13] about the protocol in Figure 9:

- The penalizing factor δ_j in Figure 9 is a $o(1)$ function and grows as $\text{poly}(n)^{-1}$.
- The *honest* prover's reward on a single threshold gate of arity k is independent of the threshold. Specifically, for a gate whose inputs have relative Hamming weight p , this honest reward—which we denote by R^{th} —is the expected value of BSR, that is $R^{\text{th}} = 2(p^2 - p + 1)$.

Campanelli-Gennaro Rational Interactive Protocol [CG15]. The protocol Π_{CG15} defines an interactive rational proof system for a low-depth Boolean function $f : \{0, 1\}^n \rightarrow \{0, 1\}$, where f is computed by a Boolean circuit C of size $|C|$, depth d , and fan-in 2. The protocol involves an interaction between a prover-verifier pair denoted by $(\mathcal{P}(x, C, f(x)), \mathcal{V}^x(C, f(x)))$ on input $x \in \{0, 1\}^n$. The interaction proceeds as follows (see Figure 10).

Initially, the prover submits a claimed output $y := f(x)$. Subsequently, the verifier engages in a recursive consistency verification: at each step, the prover provides a pair of values (y_L, y_R) corresponding to the outputs of the immediate predecessor gates feeding into a gate g , such that $g(y_L, y_R) = y$. The verifier then selects uniformly at random one of the two sub-gates and

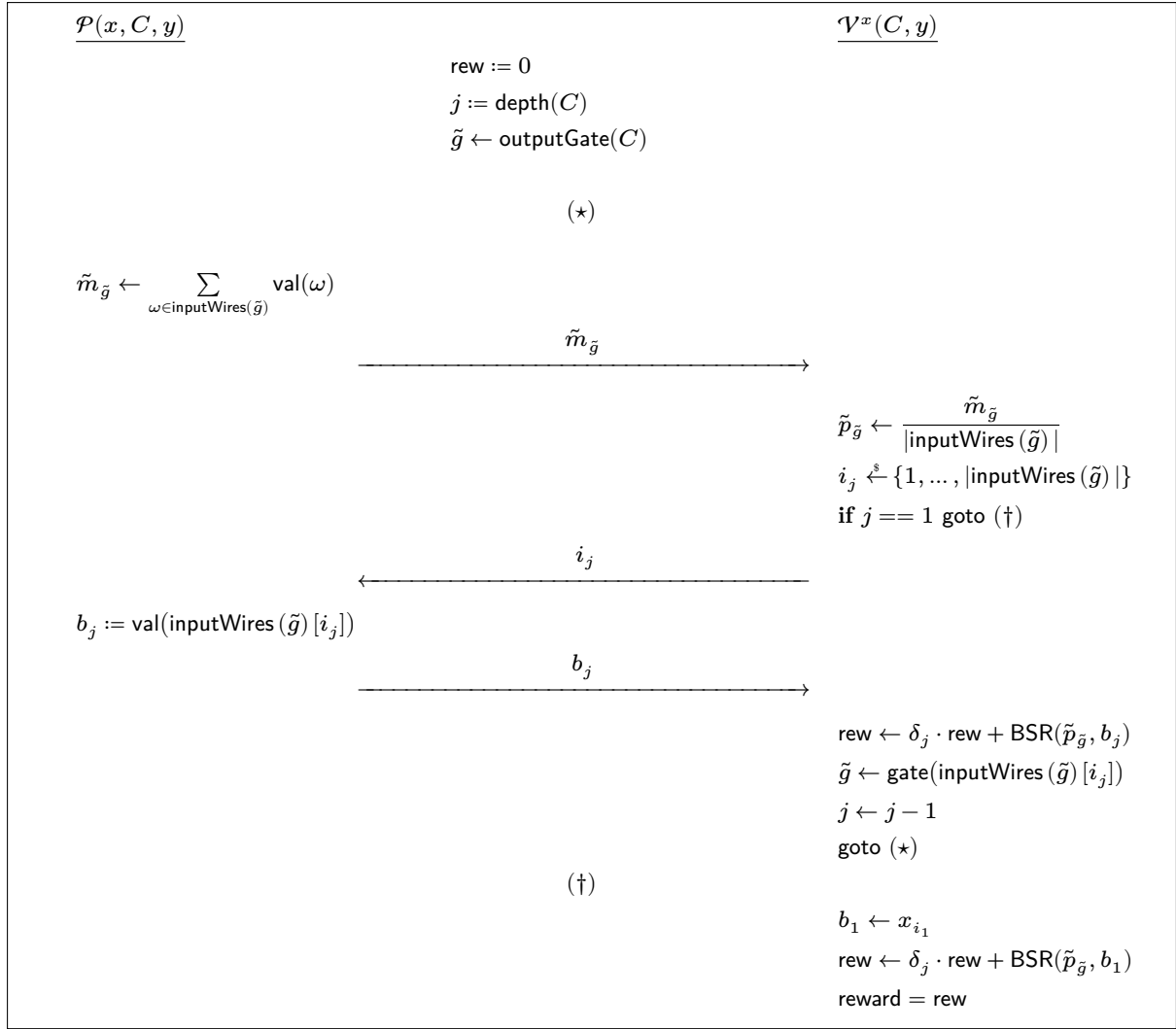


Figure 9: Consider the construction Π_{AM13} applied to a threshold circuit C of depth $\text{depth}(C)$, where the depth is defined as the number of layers of threshold gates. We assume access to a function val such that, given a wire index, it returns the value carried by that wire during the evaluation of C on input x . In addition, given a wire index, the function gate returns the threshold gate whose output corresponds to that wire. The parameter δ_j denotes a scaling factor that depends solely on the depth j of the circuit; for a detailed definition and properties of δ_j , we refer the reader to [AM13]; see also Remark 6.1.

continues the verification along the corresponding subcircuit path. This recursive verification is particularly efficient when the circuit is uniform, meaning the gate structure can be computed algorithmically in time polynomial in $\log n$. Under this uniformity assumption, the verifier's computational cost remains small.

Theorem 6.2 ([CG15]). The protocol Π_{CG15} as described in Figure 10 constitutes a secure interactive rational argument for Boolean formulas C of logarithmic depth on n -bit inputs. The protocol requires $\mathcal{O}(\log d)$ rounds of interaction. If C is T -uniform, then the verifier runs in time $\mathcal{O}(\log n \cdot T(n))$ and makes only $\mathcal{O}(1)$ queries to the input.

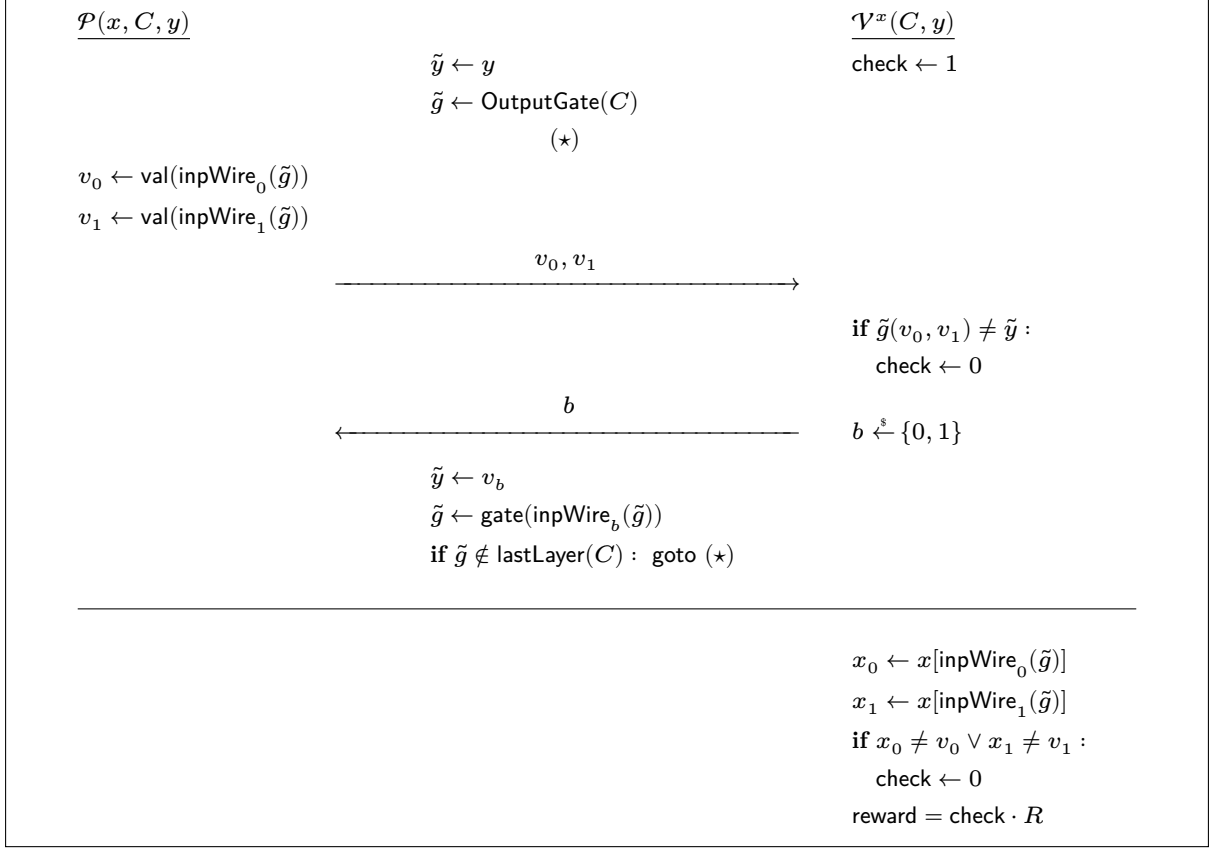


Figure 10: The construction Π_{CG15} from [CG15]. Ours is a more specific variant for a (layered) formula C of logarithmic depth. Both parties run operations placed in the center. $\text{val}(w)$ is the value of the wire w in the evaluation $C(x)$, $\text{inpWire}_0(g)$ (resp. $\text{inpWire}_1(g)$) returns the left (resp. right) input wire of gate g , $g(u, w)$ returns the output of gate g on inputs bits u, w , lastLayer is the formula's last layer of gates (whose input wires are from x). The quantity R is some fixed positive reward that depends only on the parameters.

6.2 TTF Attacks Against [CG15] and [AM13]

We now show how to instantiate the results given in § 5 to provide concrete attacks on our case studies. An intuitive characterization of the true-to-false statement attack against the protocol Π_{CG15} is provided in Figure 11.

Theorem 6.3 (True-to-False Attack on Π_{CG15}). $\text{hFS}[\Pi_{\text{CG15}}]$ is insecure.

PROOF. We instantiate the general true-to-false attack by verifying the conditions of Theorem 4.1. First, observe that the verifier in Π_{CG15} is deterministic for a given transcript τ and has a safety set $\mathcal{S} := \text{SafeSet}(\text{pp}, f, x, f(x), \tau)$. Since Π_{CG15} runs in sublinear time, we have $\ell < n$, and since the verifier must read some part of the input, $\mathcal{S} \neq \emptyset$. This satisfies the admissibility requirement of Definition 4.5.

Next, fix $f(x) = \bigwedge_{i=1}^n x_i$ and let $x = 1^n$. Then $f(x) = 1$ and $s_f(x) = n$, so f is σ -sensitive with $\sigma = n$, see Definition 5.2. By Lemma 5.3, since $\ell \geq n - \sigma + 1 = 1$, there exists $j \in \mathcal{S} \cap \text{SensitiveSet}(f, x)$. Flipping bit j yields $\hat{x} = x^{(j)}$ with $f(\hat{x}) = 0 \neq f(x)$, and since $j \in \mathcal{S}$,

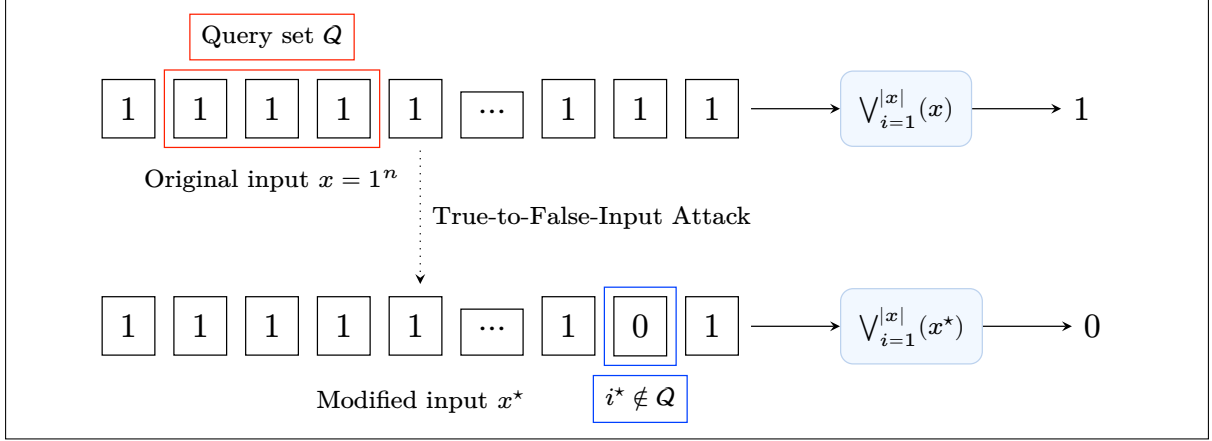


Figure 11: Example of a true-to-false-input attack on Π_{CG15} . The input x is modified at index i^* outside the verifier's query set Q , changing the function output without affecting the transcript.

the change is undetectable to the verifier. Thus, the conditions of [Theorem 5.2](#) are met, and a true-to-false adversary exists.

The result then follows directly by invoking [Lemma 4.1](#), completing the proof. $\square$

REMARK 6.2 (OR, PARITY, AND RELATED GATES). For $f(x) = \bigvee_{i=1}^n x_i$ at $x = 0^n$, $s_f(x) = n$, so the argument is identical with $\hat{x}$ obtained by flipping a single coordinate to 1. For parity $f(x) = \bigoplus_{i=1}^n x_i$, $s_f(x) = n$ for all x , so the σ -sensitivity attack applies at any input. The same reasoning extends to any fan-in-2 circuit family whose verifier induces a nonempty admissible safety set and for which there exists x with $s_f(x) \geq \sigma$; apply [Lemma 5.3](#) and then [Theorem 5.2](#).

Theorem 6.4 (True-to-False Attack on Π_{AM13}). The protocol $\text{hFS}[\Pi_{\text{AM13}}]$ is insecure.

PROOF. Let $k \in \mathbb{N}$ denote the dimension parameter, and set $n = k^2$ so that the input length grows quadratically with k . Consider the function $f : \{0, 1\}^n \rightarrow \{0, 1\}$ realized by the depth-two threshold circuit of [Figure 12](#). We view the input $x \in \{0, 1\}^n$ as a $k \times k$ matrix of bits

$$x = \begin{bmatrix} x_{1,1} & x_{1,2} & \cdots & x_{1,k} \\ x_{2,1} & x_{2,2} & \cdots & x_{2,k} \\ \vdots & \vdots & \ddots & \vdots \\ x_{k,1} & x_{k,2} & \cdots & x_{k,k} \end{bmatrix},$$

where each $x_{i,j} \in \{0, 1\}$. Each first-layer gate r_i computes the majority of row i ,

$$r_i(x) = \Theta_{\geq k/2}^{(k)}(x_{i,1}, \dots, x_{i,k}), \quad i = 1, \dots, k,$$

and the output gate is

$$f(x) = \Theta_{\geq k/2}^{(k)}(r_1(x), \dots, r_k(x)).$$

Intuition. Loosely speaking, one can notice how the parameter k controls both the circuit size and the geometry of the input. Arranging the input as a $k \times k$ matrix makes the threshold

structure transparent: each row encodes a local majority, and the top gate aggregates these local majorities. At the chosen boundary input, every row is exactly on the threshold, so flipping a single 1 in the upper-left block destabilizes both a row majority and the top majority. This creates a large sensitive set of size $n/4$, ensuring that the verifier — who only checks one row — is very likely to miss the manipulated coordinate. Thus, the construction amplifies local sensitivity into a global true-to-false vulnerability.

Fix the boundary input x as follows. For each $i \leq k/2$, set $x_{i,j} = 1$ for $j \leq k/2$ and $x_{i,j} = 0$ for $j > k/2$; for each $i > k/2$, set $x_{i,j} = 0$ for all j . Equivalently,

$$x_{i,j} = \begin{cases} 1 & \text{if } i \leq k/2 \text{ and } j \leq k/2, \\ 0 & \text{otherwise.} \end{cases}$$

A schematic $k \times k$ view (the vertical bar separates the first $k/2$ and last $k/2$ columns) is

$$x = \left[\begin{array}{ccc|ccc} 1 & \cdots & 1 & 1 & 0 & \cdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 1 & \cdots & 1 & 1 & 0 & \cdots & 0 & 0 \\ \hline 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 \\ \vdots & \ddots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ 0 & \cdots & 0 & 0 & 0 & \cdots & 0 & 0 \end{array} \right]_{k \times k}.$$

Then $r_i(x) = 1$ for $i \leq k/2$ and $r_i(x) = 0$ for $i > k/2$, so the top gate sees exactly $k/2$ ones and outputs $f(x) = 1$.

Now consider single-bit flips. If $i \leq k/2$ and $x_{i,j} = 1$, flipping this bit decreases the i -th row sum from $k/2$ to $k/2 - 1$, so r_i switches from 1 to 0. The top gate then receives only $k/2 - 1$ ones and flips from 1 to 0. Thus for $\hat{x} = x^{(i,j)}$, we have $f(\hat{x}) \neq f(x)$. No other single-bit flip changes the value of f . Hence, the sensitive set at x is precisely the set of 1's in the first $k/2$ rows, giving

$$s_f(x) = \frac{k}{2} \cdot \frac{k}{2} = \frac{k^2}{4} = \frac{n}{4}.$$

So f is σ -sensitive at x with $\sigma = n/4$; see [Definition 5.2](#).

Next, consider the verifier in Π_{AM13} . The verifier is randomized, but for any fixed transcript, the randomness chooses a single row-majority gate to inspect. With overwhelming probability, the safety set $\mathcal{S}$ excludes only the inputs of that one row and contains all remaining coordinates. Thus, we have w.o.p. $\ell = n - k$. By [Lemma 5.3](#), if $\ell \geq n - \sigma + 1 = \frac{3n}{4} + 1$, then $\mathcal{S}$ intersects the sensitive set whenever $k \geq 6$. Hence, there exists $j \in \mathcal{S} \cap \text{SensitiveSet}(f, x)$.

Define $\hat{x} = x^{(j)}$. By [Definition 5.2](#), $f(\hat{x}) \neq f(x)$. Since $j \in \mathcal{S}$, the verifier fails to detect this change (cf. [Definition 4.5](#)). This is exactly a true-to-false attack: flipping a sensitive coordinate inside the admissible safety set yields a false input accepted by the verifier. By [Theorem 5.2](#), we conclude that $\text{hFS}[\Pi_{\text{AM13}}]$ is insecure. $\square$

Concrete TTF Attacks from Impressionable Functions A particularly interesting example is the Tribes function, defined as a disjunctive normal form composed of many small AND clauses, each called a tribe. Every tribe has a width w , and there are s such tribes. The function outputs

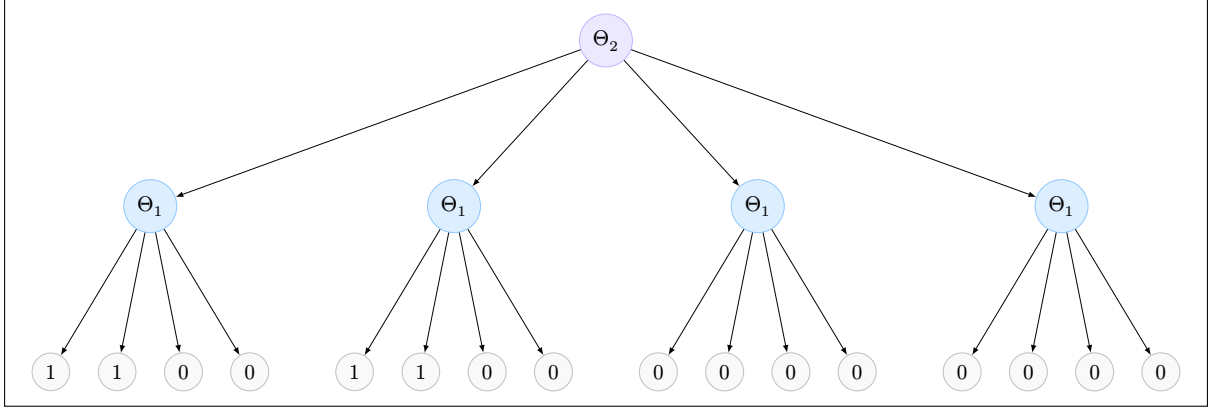


Figure 12: A depth-two threshold circuit computing the function $f : \{0, 1\}^{k \times k} \rightarrow \{0, 1\}$ in the proof of [Theorem 6.4](#) (visualized for $k = 4$). Each of the threshold gates is a majority gate. The function f is $\frac{k^2}{4}$ -sensitive.

1 precisely when at least one tribe evaluates to 1. By setting $w = \log n - \log \log n + O(1)$ and $s = n/w$, the construction becomes nearly unbiased: on a random input, the output is 1 with probability very close to $1/2$.

The influence of each variable in this case is:²⁶

$$\text{Inf}_i(\text{Tribes}_{w,s}) := \frac{\log n}{n}(1 + o(1)),$$

Both Π_{AM13} and Π_{CG15} are expressive enough to compute the Tribes function, which means we can apply [Corollary 5.1](#) to conclude attacks from different families of functions than the ones we showed so far.

7 Further Attacks Against Fiat-Shamir in the Input-Digest Model

7.1 Negative Results for AM13

We present an attack showing that the Fiat-Shamir transformation applied to the protocol Π_{AM13} fails to maintain security within the input-digest model.

Theorem 7.1. The Fiat-Shamir transform of Π_{AM13} , denoted $\text{digFS}[\Pi_{\text{AM13}}]$, is insecure in the input-digest model.

PROOF. We build an attack on a threshold circuit C^* of depth 2 (generalizable to any other depth supported by the protocol). The target circuit C^* will be a small variant of a “base” circuit $\hat{C}$, which has the following structure (see also [Figure 13](#) for a visual intuition) on input a string $x^* \in \{0, 1\}^n$ with $n = k^2$:

- all threshold gates in the circuit have arity k , which we assume to be even;

²⁶ By the KKL theorem [[KK04](#)], asymptotically this is the smallest possible maximum influence that any balanced Boolean function can achieve.

- all gates—the final output gate, as well as the k threshold gates at the intermediate layer—have threshold $\frac{k}{2}$ (they outputs 1 if $\frac{k}{2}$ or more input bits are set to 1).

The attacker selects a “base” input $\hat{x} \in \{0, 1\}^n$ structured as follows (recall that $n = k^2$):

$$\hat{x} = \overbrace{(\mathbf{1}^{k/2} \parallel \mathbf{0}^{k/2}) \parallel (\mathbf{1}^{k/2} \parallel \mathbf{0}^{k/2}) \parallel \dots \parallel (\mathbf{1}^{k/2} \parallel \mathbf{0}^{k/2})}^{k/2 \text{ times}} \parallel \mathbf{0}^{k^2/2}$$

where above we denote by $\mathbf{1}^\ell$ (resp. $\mathbf{0}^\ell$), for $\ell \in \mathbb{N}$, the repetition of 1 (resp. 0) ℓ times.

Notice that the input is intuitively split into two parts. A left-hand side where 1s and 0s are interleaved, and a right-hand side which consists of all 0s. The string $\hat{x}$ is such that all the left-hand gates at the middle-layer (the first $k/2$ gates) evaluate to 1, whereas the right-hand gates evaluate to 0.

From the above, we can make two observations, which our attacker can exploit:

- input $\hat{x}$ is such that $\hat{C}(\hat{x}) = 1$;
- changing any of the left-hand middle-layer gates²⁷ to one with threshold strictly lower than $k/2$ will cause the resulting circuit to evaluate to 0 on input $\hat{x}$.

We will show that $\mathcal{A}$ can find, with overwhelming probability, a circuit C where it will be able to claim a false output but nonetheless receive the honest reward. The pseudocode for this adversary against rational soundness in the input-digest model (Definition B.2) is below:

- (1) Let $\hat{x}$ as above, $\text{dig} := H(\hat{x})$ and $T := (\frac{k}{2} - 1, \frac{k}{2} - 2, \dots, 1, 0)$.
- (2) For each $t' \in T$:
 - Let C^* be the circuit obtained by replacing the left-most gate at the middle layer with one with threshold t' .
 - Produces a (malicious) proof for $\text{digFS}[\Pi_{\text{AM13}}]$ for the claim $C^*(\hat{x}) = 1$ as follows²⁸:
 - Let $\tilde{m}_2 = \frac{k}{2}$;
 - Let $i_2 = H(C^*, \text{dig}, \tilde{m}_2) \in [k]$;
 - If $i_2 = 1$, do not continue producing a proof for this C^* and go back to Step (2) for the next t' .
 - Otherwise (if $i_2 \neq 1$) generate b_2 and $\tilde{m}_1$ as the honest prover would.
 - Let $\pi^* = (\tilde{m}_2, i_2, b_2, \tilde{m}_1)$ and go to Step (3) skipping the next values of t' .
- (3) Return $(C^*, \hat{x}, 1, \pi^*)$

Key intuition about the attack: The attacker will reject sample a circuit until it is able to produce something that is essentially indistinguishable from the honest behavior for the evaluation of $\hat{C}(\hat{x}) = 1$ (notice that, in contrast, by construction each C^* is such that $C^*(\hat{x}) = 0$ while the adversary always falsely claim the output is 1). The only “malicious” message from the attacker is the first one, $\tilde{m}_2$. However, the only way for the reward function to be affected is if the index i_2 sampled through the RO lands on the leftmost gate (i.e., if $i_2 = 1$). This is the case that the adversary avoids.

²⁷ In our attack, we will always change the left-most one.

²⁸ Since this is a depth-2 circuit the proof is of the form $(\tilde{m}_2, i_2, b_2, \tilde{m}_1)$. See Figure 9 for reference.

If the adversary can land on some t' for which $i_2 \neq 1$, then, through the calculations below, we can show it will obtain something that in expectation (and asymptotically) is *higher* than the reward of the honest prover for the same circuit, thus breaking rational soundness. We only need to observe that $\mathcal{A}$ will be able to land on such t' with overwhelming probability. To see this is the case, observe that $|T| = \Omega(\sqrt{n})$ and that each t' will produce a uniformly distributed $i_2 \in [\sqrt{n}]$ (having only $\frac{1}{\sqrt{n}}$ probability of being equal to 1). It follows that with overwhelming probability, after at most $\approx \sqrt{n}/2$ attempts, the adversary will have found $i_2 \neq 1$.

We now argue that, for C^* found by the adversary, its reward is higher than the honest prover's. In the following calculations, we will use some ad-hoc notation: for $0 \leq \alpha \leq 1$, we denote by $R^{\text{th}}(\alpha)$ the expected reward of the honest prover in AM13 for a threshold gate with a fraction α of inputs set to 1 (see also description of the protocol earlier in this section). Note that $R^{\text{th}}(\alpha)$ is independent of the threshold of the gate (in the protocol, the verifier enforces that the claimed α of the prover is compatible with the threshold of the gate). Some notable values for R^{th} (which we will use later) are:

$$\begin{aligned} R^{\text{th}}\left(\frac{\frac{k}{2}-1}{k}\right) &= \frac{3}{2} + \frac{2}{k^2} \\ R^{\text{th}}(0) &= 2 \\ R^{\text{th}}\left(\frac{1}{2}\right) &= \frac{3}{2} \end{aligned}$$

The key difference between the honest prover and $\mathcal{A}$ is that the former will declare the correct Hamming weight of the vector of outputs of the middle layer, namely $\frac{\frac{k}{2}-1}{k}$. We remind the reader that the protocol uses a discounting factor $\delta = 1/k^u$ for some constant $u > 0$. The honest reward R_{hon} would then be:

$$R_{\text{hon}} = \delta R^{\text{th}}\left(\frac{\frac{k}{2}-1}{k}\right) + \frac{1}{2} R^{\text{th}}\left(\frac{k}{2}\right) + \frac{1}{2} R^{\text{th}}\left(\frac{k}{2}\right)$$

The adversarial prover will have “fewer branches” available (because of rejection sampling), hence the factor $\frac{k}{2} - 1$ below. We calculate $R_{\mathcal{A}}$:

$$\begin{aligned} R_{\mathcal{A}} &= \frac{\delta}{k-1} \left(\frac{k}{2} - 1\right) R^{\text{th}}\left(\frac{1}{2}\right) + \frac{1}{2} R^{\text{th}}\left(\frac{1}{2}\right) \\ &\quad + \frac{\frac{k}{2}-1}{k-1} R^{\text{th}}\left(\frac{1}{2}\right) + \frac{\frac{k}{2}}{k-1} R^{\text{th}}(0) \end{aligned}$$

Now consider their difference:

$$\begin{aligned}
R_{\mathcal{A}} - R_{\text{hon}} &= \delta \left(R^{\text{th}} \left(\frac{1}{2} \right) - R^{\text{th}} \left(\frac{k/2 - 1}{k} \right) \right) \\
&\quad + \left(\frac{k/2 - 1}{k - 1} - \frac{1}{2} \right) R^{\text{th}} \left(\frac{1}{2} \right) \\
&\quad + \left(\frac{k/2}{k - 1} - \frac{1}{2} \right) R^{\text{th}}(0) \\
&= \frac{1}{4(k - 1)} - \frac{2\delta}{k^2}
\end{aligned}$$

By observing that $\delta = 1/k^u$ for a constant $u > 0$, we can conclude that the adversarial reward is in expectation higher than the honest reward by any constant additive factor for sufficiently large input sizes. This concludes the proof. $\square$

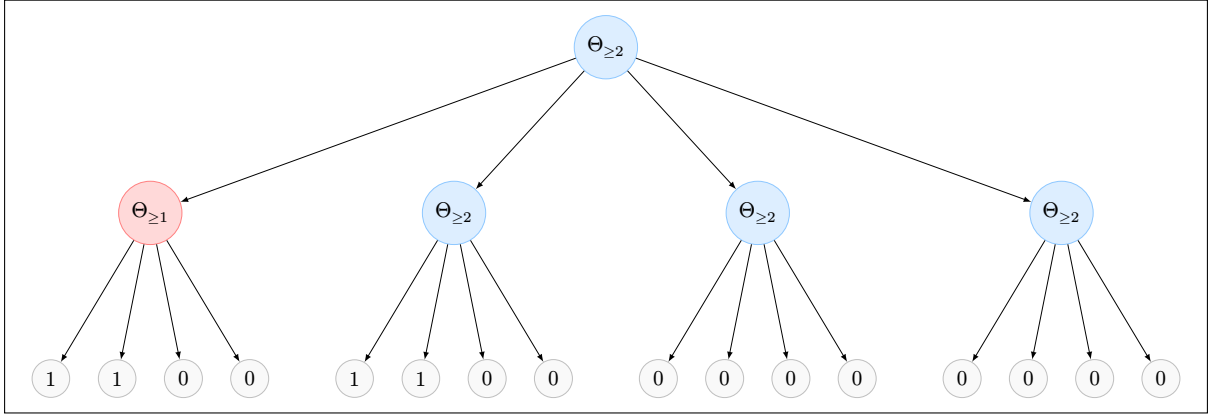


Figure 13: Simple example for the attack in [Theorem 7.1](#) (with $n = 16$, $k = 4$). On the shown input, the circuit with *only* gates of type $\Theta_{\geq 2}$ (not shown in the figure) would output 1. The adversary will transform the leftmost gates at the middle layer, switching its threshold to 1 as in figure (or any other lower threshold) for which the output 1 is false.

7.2 Negative Results for CG15

We now show that the Fiat-Shamir transform of Π_{CG15} is insecure even within the input-digest model.

Theorem 7.2. The Fiat-Shamir transform of Π_{CG15} , denoted $\text{digFS}[\Pi_{\text{CG15}}]$, is insecure in the input-digest model.

PROOF. Let $f : \{0, 1\}^n \rightarrow \{0, 1\}$ be the n -ary conjunction function defined as $f(x) := \bigwedge_{i=1}^n x_i$, and let $C^\wedge$ be the canonical balanced binary circuit computing f , consisting entirely of binary $\wedge$ gates arranged in a full binary tree. Fix the input $x := 0^{n/2} \parallel 1^{n/2}$, for which $f(x) = 0$, since the first $n/2$ bits are 0. Define $\text{digFS}[\Pi_{\text{CG15}}]$ to be the Fiat-Shamir transformation of the canonical public-coin protocol Π_{CG15} , where the verifier derives deterministic challenges by querying a

random oracle H , and receives only oracle access to x (the input-digest model). Note that the input x is fixed at the outset of the interaction.

We construct an efficient adversary $\mathcal{A}$ that forges an accepting transcript under $\text{digFS}[\Pi_{\text{CG15}}]$ while falsely claiming $f(x) = 1$. The adversary's success rests on syntactic manipulations of the circuit structure that preserve the semantic evaluation on x but bypass the verifier's path of inspection with high probability due to the behavior of the Fiat-Shamir hash.

The adversary begins by fixing the output claim to $y := 1$, which is incorrect for x , and selects a proof string $\pi := (1, 1, \dots, 1)$, in which each response is hardcoded as the constant 1. Next, the adversary perturbs $C^\wedge$ to form new circuits $C_i^\wedge$, where each $C_i^\wedge$ is derived from $C^\wedge$ by replacing a single gate g_i at depth $d = 1$ (i.e., immediately above the input layer) from a $\wedge$ to a $\vee$ gate $C_i^\wedge := C^\wedge[g_i \mapsto \vee]$. Since each g_i lies in a subtree whose inputs are all zero (i.e., the $0^{n/2}$ prefix), this modification is semantically inert $\llbracket C_i^\wedge \rrbracket(x) = \llbracket C^\wedge \rrbracket(x) = 0$. Let $S := \{C_i^\wedge \mid C_i^\wedge = C^\wedge[g_i \mapsto \vee], \llbracket C_i^\wedge \rrbracket(x) = 0\}$ denote the set of such syntactic rewritings, where the gate substitution is guaranteed not to affect the output under x .

Let $\text{dig} := H(x)$ be the Fiat-Shamir digest of the input. The adversary now iterates over all $C_i^\wedge \in S$, and for each such circuit, computes the Fiat-Shamir challenge bit $b_i^* := H(C_i^\wedge, \text{dig}, y = 1, \pi)$. If $b_i^* = 1$, the adversary outputs the forged transcript $(C, x, y, \pi) := (C_i^\wedge, x, 1, \pi)$ and halts. Otherwise, it continues to the next index i . If all $i \in S$ yield $b_i^* = 0$, it returns $\perp$.

This procedure succeeds with overwhelming probability. Each $C_i^\wedge$ is syntactically distinct due to the altered gate label, so each query $(C_i^\wedge, \text{dig}, 1, \pi)$ to H is unique. Since H is a random oracle, the values b_i^* are i.i.d. uniform in $\{0, 1\}$. Because $|S| = \Theta(n)$ (there are $\Theta(n)$ gates at depth 1 in a binary tree of n leaves), we obtain

$$\begin{aligned} \Pr[\exists i \in S \mid b_i^* = 1] &= 1 - 2^{-|S|}, \\ &\geq 1 - 2^{-n/4}, \end{aligned}$$

which is negligible for large n .

It remains to argue the correctness of the forgery. When $b_i^* = 1$, the verifier's challenge path ζ_i is derived deterministically from the Fiat-Shamir input but is syntactically agnostic to semantics. Since π is fixed and b_i^* is independent of the location of g_i , the path ζ_i has a chance of avoiding the path leading to a 0 evaluation with constant probability. In that case, the verifier inspects only subcircuits identical to those in $C^\wedge$, and hence all local checks pass. Thus, the verifier accepts the transcript, despite the claim $y = 1$ being incorrect. The expected reward assigned to the adversary is $R \cdot (1 - \text{negl}(n))$, where R is the honest reward bound, violating rational soundness, which requires a noticeable penalty for dishonest computation.

This completes the proof that $\text{digFS}[\Pi_{\text{CG15}}]$ is not rationally sound in the input-digest model. $\square$

Acknowledgments. We are grateful to Ron Rothblum for a series of insightful conversations that helped steer our attention toward a stronger variant of the Fiat-Shamir transform. We also thank Claudio Orlandi and the Crypto Group at Aarhus University for their thorough feedback on our attacks and helpful suggestions to improve the presentation. We thank Eylon Yogev and the Cryptography Group at Bar-Ilan University for their thoughtful engagement and careful questions, which clarified how rational proofs relate to general interactive proofs under the Fiat-Shamir paradigm. Matteo Campanelli thanks Pavel Hubáček for his indirect but foundational role in this work, connecting him with Agni Datta as a thesis advisor. He also acknowledges Chaya Ganesh and Rosario Gennaro for early discussions, during a separate project, which initially raised the possibility that the Fiat-Shamir transform might fail in the rational setting. Thanks to Willy Quach for pointing out other approaches to sublinear verification.

Part of this work was carried out by Agni Datta during his master’s studies at VIT Bhopal University and during his internship at the CISP Helmholtz Center for Information Security..

References

- [ABDY24] Gal Arnon, Shany Ben-David, and Eylon Yogev. Hamming weight proofs of proximity with one-sided error. In Elette Boyle and Mohammad Mahmoody, editors, *TCC 2024: 22nd Theory of Cryptography Conference, Part I*, volume 15364 of *Lecture Notes in Computer Science*, pages 125–157, Milan, Italy, December 2–6, 2024. Springer, Cham, Switzerland. doi:10.1007/978-3-031-78011-0_5. (Cited on pp. 3 and 11).
- [AM12] Pablo Daniel Azar and Silvio Micali. Rational proofs. In Howard J. Karloff and Toniann Pitassi, editors, *44th Annual ACM Symposium on Theory of Computing*, pages 1017–1028, New York, NY, USA, May 19–22, 2012. ACM Press. doi:10.1145/2213977.2214069. (Cited on pp. 3, 4, 5, 11, 14, and 49).
- [AM13] Pablo Daniel Azar and Silvio Micali. Super-efficient rational proofs. In *Proceedings of the Fourteenth Acm Conference on Electronic Commerce*, EC ’13. ACM, June 2013. doi:10/gtxc77. (Cited on pp. 2, 6, 11, 14, 32, 33, 34, and 35).
- [Bar01] Boaz Barak. How to go beyond the black-box simulation barrier. In *42nd Annual Symposium on Foundations of Computer Science*, pages 106–115, Las Vegas, NV, USA, October 14–17, 2001. IEEE Computer Society Press. doi:10.1109/SFCS.2001.959885. (Cited on pp. 3 and 11).
- [BKP18] Nir Bitansky, Yael Tauman Kalai, and Omer Paneth. Multi-collision resistance: a paradigm for keyless hash functions. In Ilias Diakonikolas, David Kempe, and Monika Henzinger, editors, *50th Annual ACM Symposium on Theory of Computing*, pages 671–684, Los Angeles, CA, USA, June 25–29, 2018. ACM Press. doi:10.1145/3188745.3188870. (Cited on p. 11).
- [BPW12] David Bernhard, Olivier Pereira, and Bogdan Warinschi. How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios. In Xiaoyun Wang and Kazue Sako, editors, *Advances in Cryptology – ASIACRYPT 2012*, volume 7658 of *Lecture Notes in Computer Science*, pages 626–643, Beijing, China, December 2–6, 2012. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-642-34961-4_38. (Cited on p. 11).
- [BR93] Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93: 1st Conference on Computer and Communications Security*, pages 62–73, Fairfax, Virginia, USA, November 3–5, 1993. ACM Press. doi:10.1145/168588.168596. (Cited on pp. 52 and 53).
- [Bri50] Glenn W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, January 1950. doi:10.1175/1520-0493(1950)0782.0.co;2. (Cited on pp. 14 and 32).
- [CCH⁺18] Ran Canetti, Yilei Chen, Justin Holmgren, Alex Lombardi, Guy N. Rothblum, and Ron D. Rothblum. Fiat-Shamir from simpler assumptions. *Cryptology ePrint Archive*, Report 2018/1004, 2018. URL: <https://eprint.iacr.org/2018/1004>. (Cited on p. 9).
- [CCR16] Ran Canetti, Yilei Chen, and Leonid Reyzin. On the correlation intractability of obfuscated pseudorandom functions. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part I*, volume 9562 of *Lecture Notes in Computer Science*, pages 389–415, Tel Aviv, Israel, January 10–13, 2016. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-49096-9_17. (Cited on p. 3).
- [CDR86] Stephen Cook, Cynthia Dwork, and Rüdiger Reischuk. Upper and lower time bounds for parallel random access machines without simultaneous writes. *SIAM Journal on Computing*, 15(1):87–97, February 1986. doi:10/dfh77z. (Cited on p. 12).
- [CG15] Matteo Campanelli and Rosario Gennaro. *Sequentially Composable Rational Proofs*, pages 270–288. Springer International Publishing, 2015. doi:10/gtx3j5. (Cited on pp. 2, 3, 6, 11, 14, 33, 34, and 35).
- [CG17] Matteo Campanelli and Rosario Gennaro. *Efficient Rational Proofs for Space Bounded Computations*, pages 53–73. Springer International Publishing, 2017. doi:10/gt7hrx. (Cited on p. 11).
- [CG18a] Matteo Campanelli and Rosario Gennaro. Fine-grained secure computation. In Amos Beimel and Stefan Dziembowski, editors, *TCC 2018: 16th Theory of Cryptography Conference, Part II*, volume 11240 of *Lecture Notes in Computer Science*, pages 66–97, Panaji, India, November 11–14, 2018. Springer, Cham, Switzerland. doi:10.1007/978-3-030-03810-6_3. (Cited on p. 11).
- [CG18b] Alessandro Chiesa and Tom Gur. Proofs of proximity for distribution testing. In Anna R. Karlin, editor, *ITCS 2018: 9th Innovations in Theoretical Computer Science Conference*, volume 94, pages 53:1–53:14, Cambridge, MA, USA, January 11–14, 2018. Leibniz International Proceedings in Informatics (LIPIcs). doi:10.4230/LIPIcs.ITCS.2018.53. (Cited on pp. 3 and 11).
- [CGG24] Matteo Campanelli, Chaya Ganesh, and Rosario Gennaro. How to make rational arguments practical and extractable. *IACR Communications in Cryptology (CiC)*, 1(1):19, 2024. doi:10.62056/a63z186bm. (Cited on pp. 11, 48, and 58).

- [CGH98] Ran Canetti, Oded Goldreich, and Shai Halevi. The random oracle methodology, revisited (preliminary version). In *30th Annual ACM Symposium on Theory of Computing*, pages 209–218, Dallas, TX, USA, May 23–26, 1998. ACM Press. doi:10.1145/276698.276741. (Cited on pp. 3 and 53).
- [CJW25] Liyan Chen, Zhengzhong Jin, and Daniel Wichs. Succinct non-interactive arguments of proximity. In Michal Koucký and Nikhil Bansal, editors, *57th Annual ACM Symposium on Theory of Computing*, pages 955–964, Prague, Czechia, June 23–27, 2025. ACM Press. doi:10.1145/3717823.3718172. (Cited on pp. 9, 10, and 54).
- [DMWG23] Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. Weak fiat-shamir attacks on modern proof systems. In *2023 IEEE Symposium on Security and Privacy*, pages 199–216, San Francisco, CA, USA, May 21–25, 2023. IEEE Computer Society Press. doi:10.1109/SP46215.2023.10179408. (Cited on pp. 3, 11, 12, and 53).
- [DNRS99] Cynthia Dwork, Moni Naor, Omer Reingold, and Larry J. Stockmeyer. Magic functions. In *40th Annual Symposium on Foundations of Computer Science*, pages 523–534, New York, NY, USA, October 17–19, 1999. IEEE Computer Society Press. doi:10.1109/SFFCS.1999.814626. (Cited on p. 11).
- [EKR04] Funda Ergün, Ravi Kumar, and Ronitt Rubinfeld. Fast approximate probabilistically checkable proofs. *Information and Computation*, 189(2):135–159, March 2004. doi:10/b84mjg. (Cited on p. 11).
- [Fis05] Marc Fischlin. Communication-efficient non-interactive proofs of knowledge with online extractors. In Victor Shoup, editor, *Advances in Cryptology – CRYPTO 2005*, volume 3621 of *Lecture Notes in Computer Science*, pages 152–168, Santa Barbara, CA, USA, August 14–18, 2005. Springer Berlin Heidelberg, Germany. doi:10.1007/11535218_10. (Cited on p. 53).
- [FS87] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *Advances in Cryptology – CRYPTO’86*, volume 263 of *Lecture Notes in Computer Science*, pages 186–194, Santa Barbara, CA, USA, August 1987. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-47721-7_12. (Cited on pp. 3, 11, 52, and 53).
- [Gen22] Gensyn. Gensyn litepaper, February 2022. URL: <https://docs.gensyn.ai/litepaper>. (Cited on p. 58).
- [GGR15] Oded Goldreich, Tom Gur, and Ron D. Rothblum. Proofs of proximity for context-free languages and read-once branching programs - (extended abstract). In Magnús M. Halldórsson, Kazuo Iwama, Naoki Kobayashi, and Bettina Speckmann, editors, *ICALP 2015: 42nd International Colloquium on Automata, Languages and Programming, Part I*, volume 9134 of *Lecture Notes in Computer Science*, pages 666–677, Kyoto, Japan, July 6–10, 2015. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-47672-7_54. (Cited on pp. 3 and 11).
- [GHRV14] Siyao Guo, Pavel Hubáček, Alon Rosen, and Margarita Vald. Rational arguments: single round delegation with sublinear verification. In Moni Naor, editor, *ITCS 2014: 5th Conference on Innovations in Theoretical Computer Science*, pages 523–540, Princeton, NJ, USA, January 12–14, 2014. Association for Computing Machinery. doi:10.1145/2554797.2554845. (Cited on pp. 3, 11, 48, and 49).
- [GHRV16] Siyao Guo, Pavel Hubáček, Alon Rosen, and Margarita Vald. Rational sumchecks. In Eyal Kushilevitz and Tal Malkin, editors, *TCC 2016-A: 13th Theory of Cryptography Conference, Part II*, volume 9563 of *Lecture Notes in Computer Science*, pages 319–351, Tel Aviv, Israel, January 10–13, 2016. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-49099-0_12. (Cited on pp. 3, 9, and 11).
- [GK03] Shafi Goldwasser and Yael Tauman Kalai. On the (in)security of the Fiat-Shamir paradigm. In *44th Annual Symposium on Foundations of Computer Science*, pages 102–115, Cambridge, MA, USA, October 11–14, 2003. IEEE Computer Society Press. doi:10.1109/SFCS.2003.1238185. (Cited on pp. 3 and 11).
- [Gol17] Oded Goldreich. *Introduction to Property Testing*. Cambridge University Press, November 2017. doi:10/ggpp77. (Cited on p. 53).
- [GR15] Tom Gur and Ron D. Rothblum. Non-interactive proofs of proximity. In Tim Roughgarden, editor, *ITCS 2015: 6th Conference on Innovations in Theoretical Computer Science*, pages 133–142, Rehovot, Israel, January 11–13, 2015. Association for Computing Machinery. doi:10.1145/2688073.2688079. (Cited on p. 54).
- [GR17] Tom Gur and Ron D. Rothblum. A hierarchy theorem for interactive proofs of proximity. In Christos H. Papadimitriou, editor, *ITCS 2017: 8th Innovations in Theoretical Computer Science Conference*, volume 4266, pages 39:1–39:43, Berkeley, CA, USA, January 9–11, 2017. Leibniz International Proceedings in Informatics (LIPIcs). doi:10.4230/LIPIcs.ITCS.2017.39. (Cited on pp. 3 and 11).
- [GR22] Guy Goldberg and Guy N. Rothblum. Sample-based proofs of proximity. In Mark Braverman, editor, *ITCS 2022: 13th Innovations in Theoretical Computer Science Conference*, volume 215, pages 77:1–77:19,

- Berkeley, CA, USA, January 31 – February 3, 2022. Leibniz International Proceedings in Informatics (LIPIcs). doi:10.4230/LIPIcs.ITCS.2022.77. (Cited on pp. 3 and 11).
- [GRS23] Oded Goldreich, Guy N. Rothblum, and Tal Skverer. On interactive proofs of proximity with proof-oblivious queries. In Yael Tauman Kalai, editor, *ITCS 2023: 14th Innovations in Theoretical Computer Science Conference*, volume 251, pages 59:1–59:16, Cambridge, MA, USA, January 10–13, 2023. Leibniz International Proceedings in Informatics (LIPIcs). doi:10.4230/LIPIcs.ITCS.2023.59. (Cited on pp. 3 and 11).
- [GW11] Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd Annual ACM Symposium on Theory of Computing*, pages 99–108, San Jose, CA, USA, June 6–8, 2011. ACM Press. doi:10.1145/1993636.1993651. (Cited on p. 5).
- [HLPT20] Thomas Haines, Sarah Jamie Lewis, Olivier Pereira, and Vanessa Teague. How not to prove your election outcome. In *2020 IEEE Symposium on Security and Privacy*, pages 644–660, San Francisco, CA, USA, May 18–21, 2020. IEEE Computer Society Press. doi:10.1109/SP40000.2020.00048. (Cited on p. 11).
- [HLR21] Justin Holmgren, Alex Lombardi, and Ron D. Rothblum. Fiat-Shamir via list-recoverable codes (or: parallel repetition of GMW is not zero-knowledge). In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd Annual ACM Symposium on Theory of Computing*, pages 750–760, Virtual Event, Italy, June 21–25, 2021. ACM Press. doi:10.1145/3406325.3451116. (Cited on p. 11).
- [IW14] Yuval Ishai and Mor Weiss. Probabilistically checkable proofs of proximity with zero-knowledge. In Yehuda Lindell, editor, *TCC 2014: 11th Theory of Cryptography Conference*, volume 8349 of *Lecture Notes in Computer Science*, pages 121–145, San Diego, CA, USA, February 24–26, 2014. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-642-54242-8_6. (Cited on pp. 3 and 11).
- [IY17] Keita Inasawa and Kenji Yasunaga. Rational proofs against rational verifiers. Cryptology ePrint Archive, Report 2017/270, 2017. URL: <https://eprint.iacr.org/2017/270>. (Cited on pp. 3 and 58).
- [JKKZ21] Ruta Jawale, Yael Tauman Kalai, Dakshita Khurana, and Rachel Yun Zhang. SNARGs for bounded depth computations and PPAD hardness from sub-exponential LWE. In Samir Khuller and Virginia Vassilevska Williams, editors, *53rd Annual ACM Symposium on Theory of Computing*, pages 708–721, Virtual Event, Italy, June 21–25, 2021. ACM Press. doi:10.1145/3406325.3451055. (Cited on p. 11).
- [Juk12] Stasys Jukna. *Boolean Function Complexity: Advances and Frontiers*. Springer Berlin Heidelberg, 2012. doi:10/fzsz2d. (Cited on p. 12).
- [Kiy22] Susumu Kiyoshima. Public-coin 3-round zero-knowledge from learning with errors and keyless multi-collision-resistant hash. In Yevgeniy Dodis and Thomas Shrimpton, editors, *Advances in Cryptology – CRYPTO 2022, Part I*, volume 13507 of *Lecture Notes in Computer Science*, pages 444–473, Santa Barbara, CA, USA, August 15–18, 2022. Springer, Cham, Switzerland. doi:10.1007/978-3-031-15802-5_16. (Cited on p. 11).
- [KK04] Claire Kenyon and Samuel Kutin. Sensitivity, block sensitivity, and ℓ -block sensitivity of boolean functions. *Information and Computation*, 189(1):43–53, February 2004. doi:10/fwzxm4. (Cited on pp. 12 and 38).
- [KLVW23] Yael Kalai, Alex Lombardi, Vinod Vaikuntanathan, and Daniel Wichs. Boosting batch arguments and RAM delegation. In Barna Saha and Rocco A. Servedio, editors, *55th Annual ACM Symposium on Theory of Computing*, pages 1545–1552, Orlando, FL, USA, June 20–23, 2023. ACM Press. doi:10.1145/3564246.3585200. (Cited on p. 28).
- [KR15] Yael Tauman Kalai and Ron D. Rothblum. Arguments of proximity - [extended abstract]. In Rosario Gennaro and Matthew J. B. Robshaw, editors, *Advances in Cryptology – CRYPTO 2015, Part II*, volume 9216 of *Lecture Notes in Computer Science*, pages 422–442, Santa Barbara, CA, USA, August 16–20, 2015. Springer Berlin Heidelberg, Germany. doi:10.1007/978-3-662-48000-7_21. (Cited on p. 54).
- [KRS25] Dmitry Khovratovich, Ron D. Rothblum, and Lev Soukhanov. How to prove false statements: Practical attacks on fiat-shamir. Cryptology ePrint Archive, Paper 2025/118, 2025. URL: <https://eprint.iacr.org/2025/118>. (Cited on p. 3).
- [Mic94] Silvio Micali. CS proofs (extended abstracts). In *35th Annual Symposium on Foundations of Computer Science*, pages 436–453, Santa Fe, NM, USA, November 20–22, 1994. IEEE Computer Society Press. doi:10.1109/SFCS.1994.365746. (Cited on p. 3).
- [PS96] David Pointcheval and Jacques Stern. Security proofs for signature schemes. In Ueli M. Maurer, editor, *Advances in Cryptology – EUROCRYPT’96*, volume 1070 of *Lecture Notes in Computer Science*, pages 387–398, Saragossa, Spain, May 12–16, 1996. Springer Berlin Heidelberg, Germany. doi:10.1007/3-540-68339-9_33. (Cited on pp. 3 and 52).

- [RR20] Guy N. Rothblum and Ron D. Rothblum. Batch verification and proofs of proximity with polylog overhead. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020: 18th Theory of Cryptography Conference, Part II*, volume 12551 of *Lecture Notes in Computer Science*, pages 108–138, Durham, NC, USA, November 16–19, 2020. Springer, Cham, Switzerland. doi:10.1007/978-3-030-64378-2_5. (Cited on pp. 3 and 11).
- [RVW13] Guy N. Rothblum, Salil P. Vadhan, and Avi Wigderson. Interactive proofs of proximity: delegating computation in sublinear time. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *45th Annual ACM Symposium on Theory of Computing*, pages 793–802, Palo Alto, CA, USA, June 1–4, 2013. ACM Press. doi:10.1145/2488608.2488709. (Cited on pp. 3, 4, 11, and 54).
- [Tha22a] Justin Thaler. Measuring SNARK performance, November 2022. URL: <https://a16zcrypto.com/posts/article/measuring-snark-performance-frontends-backends-and-the-future>. (Cited on pp. 3 and 5).
- [Tha22b] Justin Thaler. Proofs, arguments, and zero-knowledge. *Foundations and Trends® in Privacy and Security*, 4(2-4):117–660, 2022. (Cited on p. 5).
- [WB15] Michael Walfish and Andrew J Blumberg. Verifying computations without reexecuting them. *Communications of the ACM*, 58(2):74–84, 2015. (Cited on p. 3).

A Prior Definitions of Rational Proofs and Arguments

In this appendix, we formally restate the canonical definitions of rational arguments, as originally introduced by Guo, Hubáček, Rosen, and Vald [GHRV14], and subsequently reformulated in the work of Campanelli, Ganesh, and Gennaro [CGG24]. These models quantify the incentive structure of the prover through the notion of a reward gap, in contrast with the game-based definition of rational soundness presented in the main body.

A.1 Definition Based on Reward Gap

We adopt the definitional framework as specified in [GHRV14, CGG24].

Definition A.1 (Rational Argument). Let $f : \{0, 1\}^n \rightarrow \{0, 1\}^*$ be a function represented by a Boolean circuit. We say that the function f admits a rational argument if there exists an interactive protocol $\langle \mathcal{P}, \mathcal{V} \rangle$ and a reward function $\text{reward} : \{0, 1\}^* \rightarrow \mathbb{R}_{\geq 0}$, such that the following properties are satisfied for every input $x \in \{0, 1\}^n$ and every prover $\widehat{\mathcal{P}}$ of size at most $2^{\lambda(n)}$:

- **Correctness:** The verifier outputs $f(x)$ with overwhelming probability:

$$\Pr[\text{out}(\langle \mathcal{P}, \mathcal{V} \rangle(x)) = f(x)] \geq 1 - \text{negl}(\lambda),$$

where $\text{negl}(n) : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is negligible. As discussed in the main body, the verifier's output may be omitted without loss of generality.

- **Reward Bound:** The expected reward of any prover $\widehat{\mathcal{P}}$ is bounded above by that of the honest prover up to negligible additive slack:

$$\mathbb{E}[\text{reward}(\langle \widehat{\mathcal{P}}(x), \mathcal{V}^x \rangle)] \leq \mathbb{E}[\text{reward}(\langle \mathcal{P}(x), \mathcal{V}^x \rangle)] + \text{negl}(\lambda).$$

- **Noticeable Reward Gap:** The protocol needs to satisfy the reward gap condition formalized in Definition A.2.

A.2 Quantifying Adversarial Loss: Reward Gap

The reward gap quantifies the disincentive associated with dishonest computation. It formalizes the minimum expected reward loss incurred by a prover who always returns an incorrect output.

Definition A.2 (Reward Gap). Let $\langle \mathcal{P}, \mathcal{V} \rangle$ be a rational argument for a function $f : \{0, 1\}^n \rightarrow \{0, 1\}^*$ with associated reward function reward . For any prover $\widehat{\mathcal{P}}$ and input $x \in \{0, 1\}^n$, define the error probability and reward loss respectively by:

$$\epsilon_{\widehat{\mathcal{P}}} := \Pr[\text{out}(\langle \widehat{\mathcal{P}}, \mathcal{V} \rangle(x)) \neq f(x)],$$

$$\delta_{\widehat{\mathcal{P}}}(x) := \mathbb{E}[\text{reward}(\langle \mathcal{P}(x), \mathcal{V}^x \rangle)] - \mathbb{E}[\text{reward}(\langle \widehat{\mathcal{P}}(x), \mathcal{V}^x \rangle)].$$

The reward gap function $\Delta : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ is defined as:

$$\Delta(n) := \min_{x \in \{0, 1\}^n} \left(\min_{\widehat{\mathcal{P}} : \epsilon_{\widehat{\mathcal{P}}} = 1} \delta_{\widehat{\mathcal{P}}}(x) \right),$$

where the minimization is taken over all provers $\widehat{\mathcal{P}}$ that always output incorrect results.

REMARK A.1. For every prover $\widehat{\mathcal{P}}$ and input $x \in \{0, 1\}^n$, the expected reward loss satisfies the inequality

$$\delta_{\widehat{\mathcal{P}}}(x) \geq \epsilon_{\widehat{\mathcal{P}}} \cdot \Delta(x).$$

In particular, a non-negligible probability of producing incorrect outputs implies a non-negligible expected reward penalty. This inequality formalizes the disincentive to deviate from honest behavior.

We now proceed to a formal comparison between the rational argument model developed in this work and prior approaches, with particular emphasis on those presented in [Appendix A](#).

REMARK A.2 (CONCEPTUAL ADVANTAGES OF OUR [DEFINITION 3.3](#)). The notion of rational soundness introduced by [\[GHRV14\]](#) is based on the definition of a *reward gap*, which is formally the infimum, taken over all possible inputs and adversarial strategies, of the difference in expected reward between the honest prover and the adversarial prover. This formulation was developed in response to the insufficiencies in the earlier framework proposed by Azar and Micali [\[AM12\]](#), which lacks precise semantic formalization.

Both of the aforementioned definitions show fundamental limitations: the approach of [\[AM12\]](#) suffers from definitional imprecision, while the reward-gap framework of [\[GHRV14\]](#) introduces additional technical overhead and constrains the class of admissible adversaries to those with non-negligible cheating probability. On the flip side, the model proposed in this work offers several advantages. It exhibits a simpler structural formulation, conforms more closely to conventional paradigms in cryptographic soundness arguments, and does not require imposing further constraints on adversarial behavior. Likewise, as shown in [Theorem A.1](#), the proposed definition strictly generalizes and implies the reward-gap condition. In particular, the soundness guarantee remains valid against adversaries whose cheating probability is negligible, as demonstrated in [Lemma A.1](#).

A.3 Equivalence Between Models

Comparison of Rational Argument Models. The correspondence in correctness-style guarantees between our model and prior approaches is immediate. A minor syntactic distinction is that earlier models explicitly define the verifier’s output, whereas in our setting, this is omitted under the assumption that the verifier deterministically outputs the value claimed by the prover. We now formalize the equivalence of the respective soundness notions.

Theorem A.1 (Equivalence with Reward-Gap Soundness). Let Π be a rational argument system. Then Π satisfies rational soundness as defined in [Definition 3.3](#) if and only if Π achieves a noticeable reward gap as defined in [Definition A.2](#).

PROOF. Without loss of generality, assume that all reward values lie in the interval $[0, 1]$, and that the honest prover obtains an expected reward equal to 1. This assumption is without loss of generality due to normalization. Let $(\mathcal{P}, \mathcal{V})$ be a rational argument system for some function f , and let reward denote the associated reward function. Fix an input $x \in \{0, 1\}^n$, where n is polynomially bounded in the security parameter $\lambda \in \mathbb{N}$.

(FORWARD DIRECTION). Suppose that Π satisfies rational soundness as stated in [Definition 3.3](#). Then there exists a polynomially bounded function $q: \mathbb{N} \rightarrow \mathbb{R}$ such that for every PPT adversary $\mathcal{A}$, it holds that

$$\mathbb{E} [\text{ExpReward}^{\mathcal{A}}(\lambda)] \geq q(\lambda)^{-1}.$$

From the definition of the adaptive reward game (cf. [Figure 2](#)), this expectation equals $\mathbb{E}[R_{\mathcal{P}}] - \mathbb{E}[R_{\mathcal{A}}]$, where $R_{\mathcal{P}}$ and $R_{\mathcal{A}}$ denote the expected rewards of the honest and adversarial provers, respectively. In the case where the adversary outputs an incorrect answer with probability 1, we have

$$\mathbb{E}[R_{\mathcal{P}}] - \mathbb{E}[R_{\mathcal{A}}] \geq q(\lambda)^{-1}.$$

By the definition of the reward gap function $\Delta: \mathbb{N} \rightarrow \mathbb{R}$ in [Definition A.2](#), this implies that $\Delta(\lambda) \geq q(\lambda)^{-1}$, so the reward gap is noticeable.

(REVERSE DIRECTION). Suppose that Π admits a noticeable reward gap. That is, there exists a polynomially bounded function $p: \mathbb{N} \rightarrow \mathbb{R}$ such that $\Delta(\lambda) \geq p(\lambda)^{-1}$. Let $\mathcal{A}$ be an arbitrary PPT adversary, and consider the adaptive reward game $\text{ExpReward}^{\mathcal{A}}(\lambda)$. Then, by [Lemma A.1](#), there exists a derived adversary $\mathcal{B}$ that always outputs an incorrect answer, and such that $\mathbb{E}[R_{\mathcal{P}}] - \mathbb{E}[R_{\mathcal{B}}] \geq p(\lambda)^{-1}$. Hence,

$$\mathbb{E} [\text{ExpReward}^{\mathcal{A}}(\lambda)] \geq \mathbb{E} [\text{ExpReward}^{\mathcal{B}}(\lambda)] \geq p(\lambda)^{-1},$$

which shows rational soundness. This completes the proof. $\square$

Adversarial Reduction under Honest Behavior Assumption. We now present a technical lemma showing that any adversary that occasionally behaves honestly may be transformed into an adversary that always cheats, without increasing the expected reward differential.

Lemma A.1 (Cheating Reduction for Rational Soundness). Let $\mathcal{A}$ be a PPT adversary such that, for all $\lambda \in \mathbb{N}$,

$$\mathbb{E} [\text{ExpReward}^{\mathcal{A}}(\lambda)] \geq g(\lambda)$$

for some function $g: \mathbb{N} \rightarrow \mathbb{R}$. Suppose, in addition, that

$$\Pr [(f(x) = y) \wedge (f \in \mathcal{F}_{\lambda}) \mid (f, x, y) \leftarrow \mathcal{A}^H(\text{pp})] > 0.$$

Then there exists a PPT adversary $\mathcal{B}$ such that

$$\mathbb{E} [\text{ExpReward}^{\mathcal{B}}(\lambda)] \leq g(\lambda) \wedge \Pr [f(x) = y \mid (f, x, y) \leftarrow \mathcal{B}^H(\text{pp})] = 0.$$

PROOF. Define the adversary $\mathcal{B}$ as follows. On input pp , run $\mathcal{A}^H(\text{pp})$ to obtain a tuple (f, x, y, π) . If $y = f(x)$, then $\mathcal{B}$ replaces y with a fixed value $y' \neq y$ and outputs (f, x, y', π) ; otherwise, it outputs the same tuple as $\mathcal{A}$.

By construction, $\mathcal{B}$ never outputs a correct answer; hence,

$$\Pr [f(x) = y \mid (f, x, y) \leftarrow \mathcal{B}^H(\text{pp})] = 0.$$

Similarly, whenever $\mathcal{A}$ outputs a correct answer and thus obtains reward 0, the adversary $\mathcal{B}$ substitutes this with an incorrect answer, possibly obtaining a non-zero reward. Thus, it holds

pointwise that $R_{\mathcal{B}} \leq R_{\mathcal{A}}$, which implies that the game outcome $R_{\mathcal{P}} - R_{\mathcal{B}} \geq R_{\mathcal{P}} - R_{\mathcal{A}}$, and therefore

$$\mathbb{E} [\text{ExpReward}^{\mathcal{B}}(\lambda)] \leq \mathbb{E} [\text{ExpReward}^{\mathcal{A}}(\lambda)] \leq g(\lambda).$$

□

B Rational Arguments in the Input-Digest Model

We consider a syntactic and semantic variant of rational arguments in the non-interactive setting, augmented with an input digest. This model assumes the existence of a random oracle and grants the prover access to a digest of the input, which the verifier is assumed to have preprocessed once and for all. We formalize the deviation from standard definitions by explicitly introducing this additional structure.

SYNTAX B.1. A rational argument in the input-digest model is a tuple of PPT algorithms $(\text{Setup}, \mathcal{P}, \text{reward})$, each with oracle access to a random oracle H , specified as follows:

- $\text{Setup}^H(1^\lambda) \rightarrow \text{pp}$: on input the unary security parameter $\lambda \in \mathbb{N}$, outputs public parameters pp ;
- $\mathcal{P}^H(\text{pp}, x, f, y) \rightarrow \pi$: on input (pp, x, f, y) , where $x \in \{0, 1\}^*$ is the input, $f \in \mathcal{F}_\lambda$ is a function from the admissible domain, and $y \in \{0, 1\}^*$ is a claimed output, returns a proof π ;
- $\text{reward}^{x, H}(\text{pp}, f, \delta, y, \pi) \rightarrow R \in \mathbb{R}_{\geq 0}$: on input $(\text{pp}, f, \delta, y, \pi)$ with $\delta := H(x)$, outputs a non-negative real reward R .

The verifier does not receive the input x directly but is given only its digest $\delta := H(x)$. This enforces a commitment-style interface where the input is indirectly referenced via the random oracle, capturing scenarios where the verifier's access to x is to a commitment. All parties, including the prover, are assumed to have full access to H .

Definition B.1 (Rational Completeness). There exists a negligible function $\text{negl}(n) : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ such that for every PPT algorithm $\widehat{\mathcal{P}}$, for all security parameters $\lambda \in \mathbb{N}$, all inputs $x \in \{0, 1\}^*$, all functions $f \in \mathcal{F}_\lambda$, and all strings $y \in \{0, 1\}^*$, letting $\text{pp} \leftarrow \text{Setup}^H(1^\lambda)$, $\hat{\pi} \leftarrow \widehat{\mathcal{P}}(\text{pp}, x, f, y)$, and $\pi \leftarrow \mathcal{P}(\text{pp}, x, f, f(x))$, it holds that

$$\mathbb{E} [\text{reward}^x(\text{pp}, f, H(x), y, \hat{\pi})] - \mathbb{E} [\text{reward}^x(\text{pp}, f, H(x), f(x), \pi)] \leq \text{negl}(\lambda).$$

Definition B.2 (Rational Soundness in the Input-Digest Model). A rational argument in the input-digest model is said to be rationally sound with respect to a function domain $\mathcal{F} = \{\mathcal{F}_\lambda\}_{\lambda \in \mathbb{N}}$ if for every (possibly non-uniform) PPT adversary $\mathcal{A}$, there exists a polynomial $q : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ such that for all security parameters $\lambda \in \mathbb{N}$, it holds that

$$\mathbb{E} [\text{ExpReward}_{\text{dig}}^{\mathcal{A}}(\lambda)] \geq q(\lambda),$$

where the game $\text{ExpReward}_{\text{dig}}^{\mathcal{A}}$ is defined in [Figure 14](#).

We now adapt the notion of input query sets, used in constructing true-to-false-input adversaries, to the input-digest setting of rational arguments.

$\text{ExpReward}_{\text{dig}}^{\mathcal{A}}(\lambda) :$ $\begin{aligned} & \mathbf{H} \leftarrow \text{SampleRO}(1^\lambda) \\ & \mathbf{pp} \leftarrow \text{Setup}^{\mathbf{H}}(1^\lambda) \\ & (f, x, y, \pi_{\mathcal{A}}) \leftarrow \mathcal{A}^{\mathbf{H}}(\mathbf{pp}) \\ & \pi_{\mathcal{P}} \leftarrow \mathcal{P}^{\mathbf{H}}(\mathbf{pp}, f, x, f(x)) \\ & R_{\mathcal{P}} \leftarrow \text{reward}^{x, \mathbf{H}}(\mathbf{pp}, f, \mathbf{H}(x), f(x), \pi_{\mathcal{P}}) \\ & \text{if } y \neq f(x) \wedge f \in \mathcal{F}_\lambda \\ & \quad R_{\mathcal{A}} \leftarrow \text{reward}^{x, \mathbf{H}}(\mathbf{pp}, f, \mathbf{H}(x), y, \pi_{\mathcal{A}}) \\ & \text{else} \\ & \quad R_{\mathcal{A}} \leftarrow 0 \\ & \text{return } R_{\mathcal{P}} - R_{\mathcal{A}} \end{aligned}$
--

Figure 14: Reward gap game $\text{ExpReward}_{\text{dig}}^{\mathcal{A}}$ for rational soundness in the input-digest model.

Definition B.3 (Input Query Set). Let Π denote a rational argument in the input-digest model. The input query set of Π is defined as the set of pairs consisting of input indices and corresponding values queried during the execution of the reward function. Formally, given public parameters $\mathbf{pp}$, a function f , an input $x \in \{0, 1\}^*$, a claimed output $y \in \{0, 1\}^*$, and a proof π , the input query set is defined by $\text{QuerySet}_{\mathbf{pp}}(f, x, y, \pi) := \{(i, x[i]) \mid i \in \mathcal{I}\}$, where $\mathcal{I} \leftarrow \text{Query}^{x, \mathbf{H}}(\mathbf{pp}, f, \mathbf{H}(x), y, \pi)$. The dependence on $\mathbf{pp}$ may be omitted when unambiguous from context.

C Standard Fiat-Shamir Transform

We recall the classical Fiat-Shamir transform [FS87], which is a cryptographic compiler, which allows us to eliminate interaction in public-coin interactive proof systems. The transformation deterministically replaces verifier-side randomness by deriving challenge messages via oracle access to a cryptographic hash function $\mathbf{H}$, modeled as a random oracle. This abstraction, made formal in [BR93], recasts the structure of public-coin protocols into non-interactive form by substituting verifier messages with evaluations of $\mathbf{H}$ on prior transcript prefixes.

Let $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ denote an r -round public-coin interactive argument with transcript $\tau = (a_1, c_1, a_2, c_2, \dots, a_r, c_r, a_{r+1})$, where a_i is the i -th prover message and c_i is the i -th verifier challenge, sampled uniformly at random. The protocol is prover-initiated, so $c_0 := \varepsilon$, and both initiation and termination reside with the prover. The Fiat-Shamir transform deterministically defines each challenge as $c_i := \mathbf{H}(a_1, c_1, \dots, a_{i-1})$, effectively collapsing the interaction into a statically verifiable non-interactive transcript.

Analysis in the ROM. Though heuristic in the standard model, the transformation is formally justified in the ROM [BR93], where $\mathbf{H}$ is treated as a uniformly random function oracle. Within this model, the transformation has been shown to preserve soundness under structural restrictions [PS96]. The Fiat-Shamir paradigm serves as a foundational mechanism in numerous cryptographic constructions, including digital signatures and non-interactive zero-knowledge proofs.

$\text{Setup}_{\text{FS}}(1^\lambda):$	$\mathcal{P}_{\text{FS}}(\text{pp}_{\text{FS}}, x, w):$	$\mathcal{V}_{\text{FS}}(\text{pp}_{\text{FS}}, x, \pi):$
$\text{pp} \leftarrow \text{Setup}(1^\lambda)$	$(\text{pp}, H) \leftarrow \text{pp}_{\text{FS}}$	$(\text{pp}, H) \leftarrow \text{pp}_{\text{FS}}$
$H = \{H_i : \{0, 1\}^* \rightarrow \{0, 1\}^\lambda \mid i \in \mathcal{I}\}$	$\mathcal{P} \leftarrow \mathcal{P}(x, w)$	$\pi \leftarrow (a_1, \dots, a_r, a_{r+1})$
$H \stackrel{s}{\leftarrow} H_i$	for $i = 1 \dots r$	for $i = 1 \dots r$
return $\text{pp}_{\text{FS}} \leftarrow (\text{pp}, H)$	$a_i \leftarrow \mathcal{P}$	$c_i \leftarrow H(\text{pp}, x, a_1, \dots, a_i)$
	$c_i \leftarrow H(\text{pp}, x, a_1, \dots, a_i)$	return $\mathcal{V}(\text{pp}, x, (a_1, c_1, \dots, a_r, c_r, a_{r+1}))$
	$\mathcal{P} \leftarrow \mathcal{P}(c_i)$	
	$a_{r+1} \leftarrow \mathcal{P}$	
	return $\pi \leftarrow (a_1, \dots, a_r, a_{r+1})$	

Figure 15: Standard Fiat-Shamir transformation of a $(2r + 1)$ -message public-coin protocol.

We adopt the formal treatment of [DMWG23], in which the Fiat-Shamir transform is defined as a functional rewriting of transcript-generating interactive machines, preserving the round structure and public-coin semantics of the original interactive protocol Π .

Definition C.1 (Fiat-Shamir Transform). Let $\Pi = (\text{Setup}, \mathcal{P}, \mathcal{V})$ be a public-coin interactive argument of knowledge with $2r + 1$ messages for a relation $\mathcal{R}$. Let $\tau = (a_1, c_1, \dots, a_r, c_r, a_{r+1})$ denote the interaction transcript. The Fiat-Shamir transform yields a non-interactive protocol $\Pi_{\text{FS}} = (\text{Setup}_{\text{FS}}, \mathcal{P}_{\text{FS}}, \mathcal{V}_{\text{FS}})$, defined in the random oracle model as specified in Figure 15.

Random Oracle Model (ROM). The Random Oracle Model, introduced by Bellare and Rogaway [BR93], treats a cryptographic hash function as a publicly accessible random function shared among all parties. It abstracts non-interactive transformations of interactive protocols — such as Fiat-Shamir [FS87] and Fischlin [Fis05] — by modeling the oracle as a deterministic interface to a lazy-sampled random function: unseen queries are mapped to fresh uniform values, and repeated queries return consistent answers via memoization. This idealization enables provable security under programmable and sampling-based reasoning, notwithstanding known separations from the standard model [CGH98].

D Fiat-Shamir Transform for Proofs of Proximity

Before defining the Fiat-Shamir transform in the context of proximity proofs, we first revisit the foundations of proximity-based verification and its connection to classical property testing.

D.1 Property Testing and Proximity Arguments

At a high level, property testing relaxes standard decision problems by replacing exact membership with approximate distinction. Instead of asking whether an input satisfies a property exactly, the goal is to distinguish inputs that satisfy the property from those that are far from satisfying it, according to a chosen distance metric. This relaxation is powerful when the verifier has only oracle access to the input: by sampling a few coordinates, the verifier can accept valid instances and reject, with high probability, instances that deviate significantly. Classical property testers achieve this in sublinear time, often with only polylogarithmic queries [Gol17].

Not all properties admit efficient testers. To overcome these limitations, proofs of proximity enrich the verifier with an auxiliary proof. In the non-interactive setting, the verifier reads a short

proof (Merlin–Arthur proofs of proximity, MAPs). In the interactive setting, the verifier engages with an untrusted prover (interactive proofs of proximity, IPPs). These notions generalize NP and IP to the approximate regime: the verifier runs in sublinear time, queries only a small portion of the input, and nevertheless preserves soundness against adversarial provers [GR15, RVW13]. Intuitively, this enables certifying that an input is “close enough” without fully checking it.

In this work, we focus on bounded provers — computationally limited adversaries. For this reason, we use the term **proximity arguments** rather than **proximity proofs**, mirroring the distinction between arguments and proofs in complexity theory. The concept first appeared in [KR15], and has been further developed, for example, in the non-interactive setting [CJW25]. Our contribution is to adapt classical proofs of proximity into proximity arguments, enabling the use of cryptographic primitives while maintaining the sublinear-time verification paradigm.

D.2 Arguments of Proximity

To reason about proximity, we first need a way to quantify how close two strings are. The natural metric is the fraction of positions at which they differ. This measure extends to sets of strings, capturing the minimal distance of a string to the property at hand. With this in place, interactive protocols can be designed where a verifier queries only a small fraction of the input yet still decides, with high confidence, whether the input is close to the property.

Definition D.1 (Relative Hamming Distance). For $x, y \in \{0, 1\}^n$, define

$$\Delta(x, y) := \frac{1}{n} |\{i \in [n] : x_i \neq y_i\}|.$$

This lies in $[0, 1]$. If $\Delta(x, y) = 0$, the strings are identical; if $\Delta(x, y) = 1$, they differ everywhere. Given $\varepsilon \in [0, 1]$, we say x is ε -close to y if $\Delta(x, y) \leq \varepsilon$, and ε -far otherwise.

Intuitively, $\Delta(x, y)$ measures the “edit fraction” needed to transform x into y . This fractional perspective is essential: it makes sense to call a string “almost valid” when it differs from a valid one in only a small fraction of coordinates, even if exact membership fails.

Definition D.2 (Proximity to a Language). Fix $\mathcal{L} \subseteq \{0, 1\}^n$ and $\varepsilon \in [0, 1]$. For $x \in \{0, 1\}^n$, define

$$\Delta(x, \mathcal{L}) := \min_{y \in \mathcal{L}} \Delta(x, y).$$

We say x is ε -close to $\mathcal{L}$ if $\Delta(x, \mathcal{L}) \leq \varepsilon$, and ε -far otherwise.

This relaxation, from exact membership to approximate proximity, is central in property testing and enables sublinear algorithms. By allowing small errors, the verifier can examine only a few random coordinates instead of the entire input, gaining efficiency while preserving meaningful guarantees.

Definition D.3 (Interactive Argument of Proximity). An interactive argument of proximity (ARGP) for a property $\mathcal{L} \subseteq \{0, 1\}^*$ with proximity parameters $\varepsilon : \mathbb{N} \rightarrow [0, 1]$ is a tuple of PPT algorithms $(\text{Setup}, \mathcal{P}, \mathcal{V})$ defined as follows.

On input the security parameter 1^λ , the setup algorithm outputs public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. The prover $\mathcal{P}$ has $(\text{pp}, x, \mathcal{L})$ where $x \in \{0, 1\}^n$ is the input string. The verifier $\mathcal{V}$ receives $(\text{pp}, \mathcal{L})$ and oracle access to x , and engages in an interactive protocol with the prover.

The protocol satisfies:

- **Completeness.** For all $\lambda, m \in \mathbb{N}$, all $n := m\lambda$, and all $x \in \{0, 1\}^n$ such that $x \in \mathcal{L}$,

$$\Pr[\langle \mathcal{P}(\text{pp}, x, \mathcal{L}), \mathcal{V}^x(\text{pp}, \mathcal{L}) \rangle = 1] \geq 1 - \text{negl}(\lambda).$$

- **Soundness.** For all $\lambda, m \in \mathbb{N}$, all $n := m\lambda$, all $x \in \{0, 1\}^n$ such that $\Delta(x, \mathcal{L}) > \varepsilon$, and all PPT adversaries $\widehat{\mathcal{P}}$,

$$\Pr[\langle \widehat{\mathcal{P}}(\text{pp}, x, \mathcal{L}), \mathcal{V}^x(\text{pp}, \mathcal{L}) \rangle = 1] \leq \text{negl}(\lambda).$$

Probabilities are taken over the randomness of the setup, prover, and verifier (or adversary).

Interactive arguments of proximity generalize the idea of probabilistically checkable proofs, but with an explicit prover who proves closeness to the language and benefits from approximate acceptance. The verifier, equipped with oracle access to x , avoids reading the entire input by offloading the main computation to the prover while still checking proximity to the target property.

Definition D.4 (Non-Interactive Argument of Proximity (NARGP)). A non-interactive argument of proximity (NARGP) in the ROM for a property $\mathcal{L} \subseteq \{0, 1\}^*$ with proximity parameters $\varepsilon : \mathbb{N} \rightarrow [0, 1]$ is a tuple of PPT algorithms $(\text{Setup}, \mathcal{P}, \mathcal{V})$ defined as follows.

On input 1^λ , the setup algorithm outputs public parameters $\text{pp} \leftarrow \text{Setup}(1^\lambda)$. The prover receives inputs $(\text{pp}, \mathcal{L}, x)$, where $x \in \{0, 1\}^n$ with $n = m\lambda$. It also has oracle access to a random oracle H , and outputs a proof $\pi \leftarrow \mathcal{P}^{\text{H}}(\text{pp}, \mathcal{L}, x)$. The verifier receives $(\text{pp}, \mathcal{L})$, oracle access to x and to H , and outputs a decision bit $\mathcal{V}^{x, \text{H}}(\text{pp}, \mathcal{L}, \pi) \in \{0, 1\}$.

The scheme satisfies:

- **Completeness.** For all $\lambda, m \in \mathbb{N}$, $n = m\lambda$, and all $x \in \{0, 1\}^n$ such that $x \in \mathcal{L}$, we have:

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \text{H} \leftarrow \text{SampleRO}(1^\lambda) \\ \pi \leftarrow \mathcal{P}^{\text{H}}(\text{pp}, \mathcal{L}, x) \end{array} \middle| \mathcal{V}^{x, \text{H}}(\text{pp}, \mathcal{L}, \pi) = 1 \right] \geq 1 - \text{negl}(\lambda)$$

- **Soundness.** For all $\lambda, m \in \mathbb{N}$, $n = m\lambda$, all $x \in \{0, 1\}^n$ such that $\Delta(x, \mathcal{L}) > \varepsilon(n)$ and all PPT adversaries $\widehat{\mathcal{P}}$:

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \text{H} \leftarrow \text{SampleRO}(1^\lambda) \\ \pi \leftarrow \widehat{\mathcal{P}}^{\text{H}}(\text{pp}, \mathcal{L}, x) \end{array} \middle| \mathcal{V}^{x, \text{H}}(\text{pp}, \mathcal{L}, \pi) = 1 \right] \leq \text{negl}(\lambda)$$

All probabilities are taken over the randomness of the setup, the sampling of the random oracle H , and the internal randomness of the prover (in completeness) or of the adversarial prover (in soundness).

The non-interactive setting eliminates the need for back-and-forth communication. By relying on a random oracle, challenges are generated deterministically yet unpredictably, removing any opportunity for the prover to bias them. The verifier still probes only a small portion of x , but now it can do so after seeing a static proof π .

D.3 The Fiat-Shamir Transform in the Arguments of Proximity Model

We now define the Fiat-Shamir transform for arguments of proximity. As in the general setting, verifier challenges are replaced with outputs of a random oracle, so that the prover cannot bias them and the verifier can recompute them consistently. Recall, the verifier only reads sampled positions of the input x , and the transform preserves this structure: the oracle input consists of the public parameters, the language $\mathcal{L}$, and the partial transcript, while queries to x occur only after the interaction. We provide both the holographic version $\text{hFS}[\Pi]$ and the input-digest variant $\text{digFS}[\Pi]$, the latter incorporating a digest of the input x into each oracle query.

$\mathcal{P}^H(\text{pp}, x, \mathcal{L})$:	$\mathcal{V}^{H,x}(\text{pp}, \mathcal{L}, \pi)$:
for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, \mathcal{L}, m_1, \dots, m_{i-1})$ $m_i \leftarrow \mathcal{P}(e_i)$ return $\pi \leftarrow (m_1, \dots, m_r)$	$\pi \leftarrow (m_1, \dots, m_r)$ for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, \mathcal{L}, m_1, \dots, m_{i-1})$ $\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$ $I \leftarrow \text{Query}(\tau)$ $\tilde{x} \leftarrow x_I$ return $\text{Decision}^x(\text{pp}, \mathcal{L}, \tau, \tilde{x})$

Figure 16: Fiat-Shamir transform $\text{hFS}[\Pi]$ for arguments of proximity.

Definition D.5 (Fiat-Shamir for Arguments of Proximity). Let Π be an argument of proximity with r challenge rounds. In the random-oracle model, we define $\text{hFS}[\Pi] = (\mathcal{P}^H, \mathcal{V}^{H,x})$ as in Figure 16, where the prover uses H to derive challenges and the verifier recomputes them.

$\mathcal{P}^H(\text{pp}, x, \mathcal{L})$:	$\mathcal{V}^{H,x}(\text{pp}, \mathcal{L}, \pi)$:
$\text{dig} \leftarrow H(x)$ for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, \mathcal{L}, \text{dig}, m_1, \dots, m_{i-1})$ $m_i \leftarrow \mathcal{P}(e_i)$ return $\pi \leftarrow (m_1, \dots, m_r)$	$\pi \leftarrow (m_1, \dots, m_r)$ $\text{dig} \leftarrow H(x)$ for $i = 1, \dots, r$ $e_i \leftarrow H(\text{pp}, \mathcal{L}, \text{dig}, m_1, \dots, m_{i-1})$ $\tau \leftarrow (e_1, m_1, \dots, e_r, m_r)$ $I \leftarrow \text{Query}(\tau)$ $\tilde{x} \leftarrow x_I$ return $\text{Decision}^x(\text{pp}, \mathcal{L}, \text{dig}, \tau, \tilde{x})$

Figure 17: Fiat-Shamir transform with input digest $\text{digFS}[\Pi]$ for arguments of proximity. The differences from the standard transform $\text{hFS}[\Pi]$ (Figure 16) are highlighted in purple.

Definition D.6 (Input-Digest Fiat–Shamir for Arguments of Proximity). Let Π be an argument of proximity with r challenge rounds. In the random-oracle model, we define $\text{digFS}[\Pi] = (\mathcal{P}^H, \mathcal{V}^{H,x})$ as in Figure 17. The prover first computes an input digest $\text{dig} = H(x)$ and then uses H to derive challenges, incorporating dig in each query. The verifier validates the same digest, derives challenges consistently, and obtains the transcript τ .

D.4 Safe Sets in Arguments of Proximity

In this appendix, we formalize the notions of input query sets, safe sets, and true-to-false-input adversaries in the setting of arguments of proximity. These definitions mirror those used for rational arguments but are adapted to the proximity model.

Definition D.7 (Input Query Set for Proximity). Let Π be an argument of proximity for $\mathcal{L} \subseteq \{0, 1\}^*$. For pp and transcript τ , the Input Query Set is

$$\text{QuerySet}_{\text{pp}}(\mathcal{L}, \tau) := \left\{ i \in \{1, \dots, n\} \mid \Pr[i \in \text{Query}(\tau, \mathcal{L})] = 1 - \text{negl}(\lambda) \right\},$$

where $n = |x|$.

Definition D.8 (Safe Set for Proximity). The Safe Set is

$$\text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau) := \left\{ i \in \{1, \dots, n\} \mid \Pr[i \in \text{Query}(\tau, \mathcal{L})] = \text{negl}(\lambda) \right\}.$$

These are the unchecked positions where deviations may remain invisible.

Definition D.9 (Safe Set Size in Proximity). The protocol has a safe set bounded by $\ell : \mathbb{N} \rightarrow \mathbb{N}$ if for every $\lambda \in \mathbb{N}$, every n , and every $x \in \{0, 1\}^n$,

$$\Pr \left[|\mathcal{S}| \geq \ell(n) \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \tau \leftarrow \mathcal{P}(\text{pp}, x, \mathcal{L}) \\ \mathcal{S} \leftarrow \text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

This ensures that the verifier’s blind spots remain large even in the proximity setting.

Definition D.10 (True-to-False-Input Adversary in Proximity). Let Π be an argument of proximity for $\mathcal{L}$ with soundness error ε . We say it admits a True-to-False-Input Adversary if there exist PPT algorithms $(\mathcal{A}_0, \mathcal{A}_1)$ such that for every $\lambda \in \mathbb{N}$ and input length n ,

$$\Pr \left[\begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ x \leftarrow \mathcal{A}_0(1^\lambda) \\ \tau \leftarrow \mathcal{P}(\text{pp}, x, \mathcal{L}) \\ \hat{x} \leftarrow \mathcal{A}_1(\text{pp}, \mathcal{L}, x, \tau) \end{array} \mid \begin{array}{l} x \in \mathcal{L} \\ \wedge \Delta(\hat{x}, \mathcal{L}) > \varepsilon(n) \\ \wedge \Delta \mathcal{S}(x, \hat{x}) \subseteq \text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

Here, the adversary transforms a valid input into one far from $\mathcal{L}$, yet the verifier accepts because the tampering lies within the safe set.

Definition D.11 (Admissible Safety Set for Proximity). Let Π be an argument of proximity for a language $\mathcal{L} \subseteq \{0, 1\}^*$. We say that the protocol has an **admissible safety set** if there exists a deterministic algorithm $\text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau)$ running in time $\text{poly}(n)$, where n is the input length, such that for every $\lambda \in \mathbb{N}$ and every input $x \in \{0, 1\}^n$,

$$\Pr \left[|S| \neq \emptyset \mid \begin{array}{l} \text{pp} \leftarrow \text{Setup}(1^\lambda) \\ \tau \leftarrow \mathcal{P}(\text{pp}, x, \mathcal{L}) \\ S \leftarrow \text{SafeSet}_{\text{pp}}(\mathcal{L}, \tau) \end{array} \right] \geq 1 - \text{negl}(\lambda).$$

This mirrors the rational-proof notion but adapts it to the proximity setting.

Corollary D.1. If an argument of proximity Π has a deterministic verifier that queries a sublinear number of input positions, then it admits an admissible safety set as in [Definition D.11](#).

PROOF. A deterministic sublinear verifier necessarily ignores at least one coordinate, giving rise to a nonempty safe set. $\square$

E The Synergy Rational Proofs $\leftrightarrow$ Smart Contracts

When Azar and Micali introduced rational proofs in 2012, they assumed the verifier would follow the protocol and issue the reward. This assumption does not hold universally. A dishonest or rational verifier may bias randomness or abort the protocol. Azar and Micali left open the problem of *enforcing* the verifier’s behavior.²⁹

Smart contracts offer a solution. These are programs running on decentralized computers with built-in primitives for transferring assets and enforcing agreements.

This synergy is mutual. Rational proofs can be enforced via smart contracts, and their efficiency makes them particularly suited to such settings. For example, a contract may release funds if a user produces y such that $y = f(x)$; instead of verifying $y = f(x)$ directly, the user submits a rational proof. The contract then verifies the proof and executes the payment.³⁰

The Challenge of Non-Interaction A critical obstacle to applying rational arguments in smart contracts is their reliance on interaction. Known constructions require multiple rounds for non-trivial computations. But interaction is expensive or infeasible in distributed ledger settings. Non-interactive protocols reduce latency and require only one-time participation from the prover.

²⁹ To our knowledge, only [\[IY17\]](#) studies the case of a rational verifier, limited to specific threshold circuits.

³⁰ Such scenarios are actively considered by several organizations. One example is when x is data for classification, f is a machine learning model, and y is the output. The gensyn project explores such applications [\[Gen22\]](#). For a broader discussion, see [\[CGG24\]](#).